



Chapter 1

Linear Algebra and Optimization: An Introduction

“No matter what engineering field you’re in, you learn the same basic science and mathematics. And then maybe you learn a little bit about how to apply it.”—Noam Chomsky

1.1 Introduction

Machine learning builds mathematical models from data containing multiple *attributes* (i.e., variables) in order to predict some variables from others. For example, in a cancer prediction application, each data point might contain the variables obtained from running clinical tests, whereas the predicted variable might be a binary diagnosis of cancer. Such models are sometimes expressed as linear and nonlinear relationships between variables. These relationships are discovered in a data-driven manner by *optimizing* (maximizing) the “agreement” between the models and the observed data. This is an optimization problem.

Linear algebra is the study of linear operations in *vector spaces*. An example of a vector space is the infinite set of all possible *Cartesian coordinates* in two dimensions in relation to a fixed point referred to as the *origin*, and each *vector* (i.e., a 2-dimensional coordinate) can be viewed as a member of this set. This abstraction fits in nicely with the way data is represented in machine learning as points with multiple dimensions, albeit with dimensionality that is usually greater than 2. These dimensions are also referred to as attributes in machine learning parlance. For example, each patient in a medical application might be represented by a vector containing many attributes, such as age, blood sugar level, inflammatory markers, and so on. It is common to apply linear functions to these high-dimensional vectors in many application domains in order to extract their analytical properties. The study of such linear transformations lies at the heart of linear algebra.

While it is easy to visualize the spatial geometry of points/operations in 2 or 3 dimensions, it becomes harder to do so in higher dimensions. For example, it is simple to visualize

a 2-dimensional rotation of an object, but it is hard to visualize a 20-dimensional object and its corresponding rotation. This is one of the primary challenges associated with linear algebra. However, with some practice, one can transfer spatial intuitions to higher dimensions. Linear algebra can be viewed as a generalized form of the geometry of Cartesian coordinates in d dimensions. Just as one can use analytical geometry in two dimensions in order to find the intersection of two lines in the plane, one can generalize this concept to any number of dimensions. The resulting method is referred to as *Gaussian elimination* for solving systems of equations, and it is one of the fundamental cornerstones of linear algebra. Indeed, the problem of *linear regression*, which is fundamental to linear algebra, optimization, and machine learning, is closely related to solving systems of equations. This book will introduce linear algebra and optimization with a specific focus on machine learning applications.

This chapter is organized as follows. The next section introduces the definitions of vectors and matrices and important operations. Section 1.3 closely examines the nature of matrix multiplication with vectors and its interpretation as the composition of simpler transformations on vectors. In Section 1.4, we will introduce the basic problems in machine learning that are used as application examples throughout this book. Section 1.5 will introduce the basics of optimization, and its relationship with the different types of machine learning problems. A summary is given in Section 1.6.

1.2 Scalars, Vectors, and Matrices

We start by introducing the notions of scalars, vectors, and matrices, which are the fundamental structures associated with linear algebra.

1. *Scalars*: Scalars are individual numerical values that are typically drawn from the real domain in most machine learning applications. For example, the value of an attribute such as *Age* in a machine learning application is a scalar.
2. *Vectors*: Vectors are arrays of numerical values (i.e., arrays of scalars). Each such numerical value is also referred to as a *coordinate*. The individual numerical values of the arrays are referred to as *entries*, *components*, or *dimensions* of the vector, and the number of components is referred to as the vector *dimensionality*. In machine learning, a vector might contain components (associated with a data point) corresponding to numerical values like *Age*, *Salary*, and so on. A 3-dimensional vector representation of a 25-year-old person making 30 dollars an hour, and having 5 years of experience might be written as the array of numbers [25, 30, 5].
3. *Matrices*: Matrices can be viewed as rectangular arrays of numerical values containing both *rows* and *columns*. In order to access an element in the matrix, one must specify its row index and its column index. For example, consider a data set in a machine learning application containing d properties of n individuals. Each individual is allocated a row, and each property is allocated in column. In such a case, we can define a data matrix, in which each row is a d -dimensional vector containing the properties of one of the n individuals. The *size* of such a matrix is denoted by the notation $n \times d$. An element of the matrix is accessed with the pair of indices (i, j) , where the first element i is the row index, and the second element j is the column index. The row index increases from top to bottom, whereas the column index increases from left to right. The value of the (i, j) th entry of the matrix is therefore equal to the j th property of the i th individual. When we define a matrix $A = [a_{ij}]$, it refers to the fact

that the (i, j) th element of A is denoted by a_{ij} . Furthermore, defining $A = [a_{ij}]_{n \times d}$ refers to the fact that the size of A is $n \times d$. When a matrix has the same number of rows as columns, it is referred to as a *square matrix*. Otherwise, it is referred to as a *rectangular matrix*. A rectangular matrix with more rows than columns is referred to as *tall*, whereas a matrix with more columns than rows is referred to as *wide* or *fat*.

It is possible for scalars, vectors, and matrices to contain complex numbers. This book will occasionally discuss complex-valued vectors when they are relevant to machine learning.

Vectors are special cases of matrices, and scalars are special cases of both vectors and matrices. For example, a scalar is sometimes viewed as a 1×1 “matrix.” Similarly, a d -dimensional vector can be viewed as a $1 \times d$ matrix when it is treated as a *row vector*. It can also be treated as a $d \times 1$ matrix when it is a *column vector*. The addition of the word “row” or “column” to the vector definition is indicative of whether that vector is naturally a row of a larger matrix or whether it is a column of a larger matrix. By default, vectors are assumed to be column vectors in linear algebra, unless otherwise specified. We always use an overbar on a variable to indicate that it is a vector, although we do not do so for matrices or scalars. For example, the row vector $[y_1, \dots, y_d]$ of d values can be denoted by $\bar{y}$ or $\bar{Y}$. In this book, scalars are always represented by lower-case variables like a or δ , whereas matrices are always represented by upper-case variables like A or Δ .

In the sciences, a vector is often geometrically visualized as a quantity, such as the velocity, that has a magnitude as well as a direction. Such vectors are referred to as *geometric vectors*. For example, imagine a situation where the positive direction of the X -axis corresponds to the eastern direction, and the positive direction of the Y -axis corresponds to the northern direction. Then, a person that is simultaneously moving at 4 meters/second in the eastern direction and at 3 meters/second in the northern direction is really moving in the north-eastern direction in a straight line at $\sqrt{4^2 + 3^2} = 5$ meters/second (based on the *Pythagorean theorem*). This is also the length of the vector. The vector of the velocity of this person can be written as a directed line from the origin to $[4, 3]$. This vector is shown in Figure 1.1(a). In this case, the *tail* of the vector is at the origin, and the *head* of the vector is at $[4, 3]$. Geometric vectors in the sciences are allowed to have arbitrary tails. For example, we have shown another example of the same vector $[4, 3]$ in Figure 1.1(a) in which the tail is placed at $[1, 4]$ and the head is placed at $[5, 7]$. In contrast to geometric vectors, only vectors that have tails at the origin are considered in linear algebra (although the mathematical results, principles, and intuition remain the same). This does not lead to any loss of expressivity. *All vectors, operations, and spaces in linear algebra use the origin as an important reference point.*

1.2.1 Basic Operations with Scalars and Vectors

Vectors of the same dimensionality can be added or subtracted. For example, consider two d -dimensional vectors $\bar{x} = [x_1 \dots x_d]$ and $\bar{y} = [y_1 \dots y_d]$ in a retail application, where the i th component defines the volume of sales for the i th product. In such a case, the vector of aggregate sales is $\bar{x} + \bar{y}$, and its i th component is $x_i + y_i$:

$$\bar{x} + \bar{y} = [x_1 \dots x_d] + [y_1 \dots y_d] = [x_1 + y_1 \dots x_d + y_d]$$

Vector subtraction is defined in the same way:

$$\bar{x} - \bar{y} = [x_1 \dots x_d] - [y_1 \dots y_d] = [x_1 - y_1 \dots x_d - y_d]$$

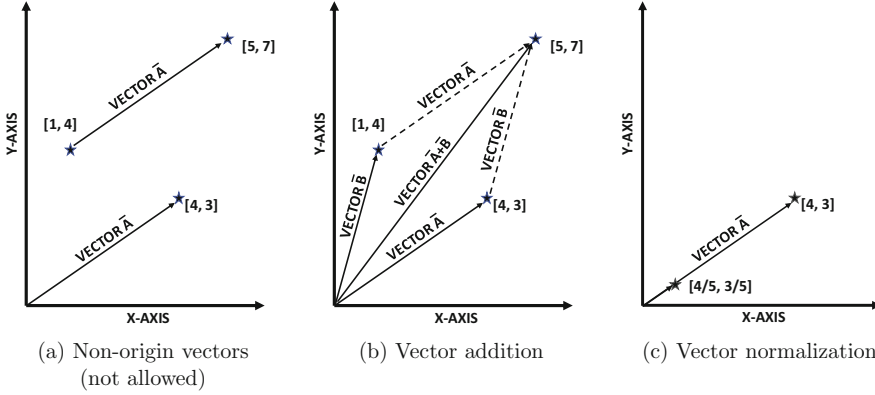


Figure 1.1: Examples of vector definition and basic operations

Vector addition is commutative (like scalar addition) because $\bar{x} + \bar{y} = \bar{y} + \bar{x}$. When two vectors, $\bar{x}$ and $\bar{y}$, are added, the origin, $\bar{x}$, $\bar{y}$, and $\bar{x} + \bar{y}$ represent the vertices of a parallelogram. For example, consider the vectors $\bar{A} = [4, 3]$ and $\bar{B} = [1, 4]$. The sum of these two vectors is $\bar{A} + \bar{B} = [5, 7]$. The addition of these two vectors is shown in Figure 1.1(b). It is easy to show that the four points $[0, 0]$, $[4, 3]$, $[1, 4]$, and $[5, 7]$ form a parallelogram in 2-dimensional space, and the addition of the vectors is one of the diagonals of the parallelogram. The other diagonal can be shown to be parallel to either $\bar{A} - \bar{B}$ or $\bar{B} - \bar{A}$, depending on the direction of the vector. Note that vector addition and subtraction follow the same rules in linear algebra as for geometric vectors, except that the tails of the vectors are always origin rooted. For example, the vector $(\bar{A} - \bar{B})$ should no longer be drawn as a diagonal of the parallelogram, but as an origin-rooted vector with the same direction as the diagonal. Nevertheless, the diagonal abstraction still helps in the computation of $(\bar{A} - \bar{B})$. One way of visualizing vector addition (in terms of the velocity abstraction) is that if a platform moves on the ground with velocity $[1, 4]$, and if the person walks on the platform (relative to it) with velocity $[4, 3]$, then the overall velocity of the person relative to the ground is $[5, 7]$.

It is possible to multiply a vector with a scalar by multiplying each component of the vector with the scalar. Consider a vector $\bar{x} = [x_1, \dots, x_d]$, which is scaled by a factor of a :

$$\bar{x}' = a\bar{x} = [a x_1 \dots a x_d]$$

For example, if the vector $\bar{x}$ contains the number of units sold of each product, then one can use $a = 10^{-6}$ to convert units sold into number of *millions of* units sold. The scalar multiplication operation simply scales the *length* of the vector, but does not change its *direction* (i.e., relative values of different components). The notion of “length” is defined more formally in terms of the norm of the vector, which is discussed below.

Vectors can be multiplied with the notion of the *dot product*. The dot product between two vectors, $\bar{x} = [x_1, \dots, x_d]$ and $\bar{y} = [y_1, \dots, y_d]$, is the sum of the element-wise multiplication of their individual components. The dot product of $\bar{x}$ and $\bar{y}$ is denoted by $\bar{x} \cdot \bar{y}$ (with a dot in the middle) and is formally defined as follows:

$$\bar{x} \cdot \bar{y} = \sum_{i=1}^d x_i y_i \quad (1.1)$$

Consider a case where we have $\bar{x} = [1, 2, 3]$ and $\bar{y} = [6, 5, 4]$. In such a case, the dot product of these two vectors can be computed as follows:

$$\bar{x} \cdot \bar{y} = (1)(6) + (2)(5) + (3)(4) = 28 \quad (1.2)$$

The dot product is a special case of a more general operation, referred to as the *inner product*, and it preserves many fundamental rules of Euclidean geometry. The space of vectors that includes a dot product operation is referred to as a *Euclidean space*. The dot product is a commutative operation:

$$\bar{x} \cdot \bar{y} = \sum_{i=1}^d x_i y_i = \sum_{i=1}^d y_i x_i = \bar{y} \cdot \bar{x}$$

The dot product also inherits the distributive property of scalar multiplication:

$$\bar{x} \cdot (\bar{y} + \bar{z}) = \bar{x} \cdot \bar{y} + \bar{x} \cdot \bar{z}$$

The dot product of a vector, $\bar{x} = [x_1, \dots, x_d]$, with itself is referred to as its squared *norm* or Euclidean norm. The norm defines the vector length and is denoted by $\|\cdot\|$:

$$\|\bar{x}\|^2 = \bar{x} \cdot \bar{x} = \sum_{i=1}^d x_i^2$$

The norm of the vector is the Euclidean distance of its coordinates from the origin. In the case of Figure 1.1(a), the norm of the vector $[4, 3]$ is $\sqrt{4^2 + 3^2} = 5$. Often, vectors are *normalized* to unit length by dividing them with their norm:

$$\bar{x}' = \frac{\bar{x}}{\|\bar{x}\|} = \frac{\bar{x}}{\sqrt{\bar{x} \cdot \bar{x}}}$$

Scaling a vector by its norm does not change the relative values of its components, which define the direction of the vector. For example, the Euclidean distance of $[4, 3]$ from the origin is 5. Dividing each component of the vector by 5 results in the vector $[4/5, 3/5]$, which changes the length of the vector to 1, but not its direction. This shortened vector is shown in Figure 1.1(c), and it overlaps with the vector $[4, 3]$. The resulting vector is referred to as a *unit vector*.

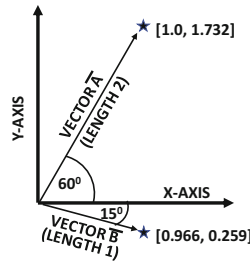
A generalization of the Euclidean norm is the L_p -norm, which is denoted by $\|\cdot\|_p$:

$$\|\bar{x}\|_p = \left(\sum_{i=1}^d |x_i|^p \right)^{(1/p)} \quad (1.3)$$

Here, $|\cdot|$ indicates the absolute value of a scalar, and p is a positive integer. For example, when p is set to 1, the resulting norm is referred to as the Manhattan norm or the L_1 -norm.

The (squared) Euclidean distance between $\bar{x} = [x_1, \dots, x_d]$ and $\bar{y} = [y_1, \dots, y_d]$ can be shown to be the dot product of $\bar{x} - \bar{y}$ with itself:

$$\|\bar{x} - \bar{y}\|^2 = (\bar{x} - \bar{y}) \cdot (\bar{x} - \bar{y}) = \sum_{i=1}^d (x_i - y_i)^2 = \text{Euclidean}(\bar{x}, \bar{y})^2$$

Figure 1.2: The angular geometry of vectors $\bar{A}$ and $\bar{B}$

Dot products satisfy the *Cauchy-Schwarz inequality*, according to which the dot product between a pair of vectors is bounded above by the product of their lengths:

$$\left| \sum_{i=1}^d x_i y_i \right| = |\bar{x} \cdot \bar{y}| \leq \|\bar{x}\| \|\bar{y}\| \quad (1.4)$$

The Cauchy-Schwarz inequality can be proven by first showing that $|\bar{x} \cdot \bar{y}| \leq 1$ when $\bar{x}$ and $\bar{y}$ are unit vectors (i.e., the result holds when the arguments are unit vectors). This is because both $\|\bar{x} - \bar{y}\|^2 = 2 - 2\bar{x} \cdot \bar{y}$ and $\|\bar{x} + \bar{y}\|^2 = 2 + 2\bar{x} \cdot \bar{y}$ are nonnegative. This is possible only when $|\bar{x} \cdot \bar{y}| \leq 1$. One can then generalize this result to arbitrary length vectors by observing that the dot product scales up linearly with the norms of the underlying arguments. Therefore, one can scale up both sides of the inequality with the norms of the vectors.

Problem 1.2.1 (Triangle Inequality) Consider the triangle formed by the origin, $\bar{x}$, and $\bar{y}$. Use the Cauchy-Schwarz inequality to show that the side length $\|\bar{x} - \bar{y}\|$ is no greater than the sum $\|\bar{x}\| + \|\bar{y}\|$ of the other two sides.

A hint for solving the above problem is that both sides of the triangle inequality are non-negative. Therefore, the inequality is true if and only if it holds after squaring both sides.

The Cauchy-Schwarz inequality shows that the dot product between a pair of vectors is no greater than the product of vector lengths. In fact, the ratio between these two quantities is the cosine of the angle between the two vectors (which is always less than 1). For example, one often represents the coordinates of a 2-dimensional vector in *polar form* as $[a, \theta]$, where a is the length of the vector, and θ is the counter-clockwise angle the vector makes with the X-axis. The Cartesian coordinates are $[a \cos(\theta), a \sin(\theta)]$, and the dot product of this Cartesian coordinate vector with $[1, 0]$ (the X-axis) is $a \cos(\theta)$. As another example, consider two vectors with lengths 2 and 1, respectively, which make (counter-clockwise) angles of 60° and -15° with respect to the X-axis in a 2-dimensional setting. These vectors are shown in Figure 1.2. The coordinates of these vectors are $[2 \cos(60), 2 \sin(60)] = [1, \sqrt{3}]$ and $[\cos(-15), \sin(-15)] = [0.966, -0.259]$.

The cosine function between two vectors $\bar{x} = [x_1 \dots x_d]$ and $\bar{y} = [y_1 \dots y_d]$ is algebraically defined by the dot product between the two vectors after scaling them to unit norm:

$$\cos(\bar{x}, \bar{y}) = \frac{\bar{x} \cdot \bar{y}}{\sqrt{\bar{x} \cdot \bar{x}} \sqrt{\bar{y} \cdot \bar{y}}} = \frac{\bar{x} \cdot \bar{y}}{\|\bar{x}\| \|\bar{y}\|} \quad (1.5)$$

The algebraically computed cosine function over $\bar{x}$ and $\bar{y}$ has the normal trigonometric interpretation of being equal to $\cos(\theta)$, where θ is the angle between the vectors $\bar{x}$ and $\bar{y}$.

For example, the two vectors $\bar{A}$ and $\bar{B}$ in Figure 1.2 are at an angle of 75° to each other, and have norms of 1 and 2, respectively. Then, the algebraically computed cosine function over the pair $[\bar{A}, \bar{B}]$ is equal to the expected trigonometric value of $\cos(75)$:

$$\cos(\bar{A}, \bar{B}) = \frac{0.966 \times 1 - 0.259 \times \sqrt{3}}{1 \times 2} \approx 0.259 \approx \cos(75)$$

In order to understand why the algebraic dot product between two vectors yields the trigonometric cosine value, one can use the *cosine law* from Euclidean geometry. Consider the triangle created by the origin, $\bar{x} = [x_1, \dots, x_d]$ and $\bar{y} = [y_1, \dots, y_d]$. We want to find the angle θ between $\bar{x}$ and $\bar{y}$. The Euclidean side lengths of this triangle are $a = \|\bar{x}\|$, $b = \|\bar{y}\|$, and $c = \|\bar{x} - \bar{y}\|$. The cosine law provides a formula for the angle θ in terms of side lengths as follows:

$$\cos(\theta) = \frac{a^2 + b^2 - c^2}{2ab} = \frac{\|\bar{x}\|^2 + \|\bar{y}\|^2 - \|\bar{x} - \bar{y}\|^2}{2(\|\bar{x}\|)(\|\bar{y}\|)} = \frac{\bar{x} \cdot \bar{y}}{\sqrt{\bar{x} \cdot \bar{x}} \sqrt{\bar{y} \cdot \bar{y}}}$$

The second relationship is obtained by expanding $\|\bar{x} - \bar{y}\|^2$ as $(\bar{x} - \bar{y}) \cdot (\bar{x} - \bar{y})$ and then using the distributive property of dot products. Almost all the wonderful geometric properties of Euclidean spaces can be algebraically traced back to this simple relationship between the dot product and the trigonometric cosine. *The simple algebra of the dot product operation hides a lot of complex Euclidean geometry.* The exercises at the end of this chapter show that many basic geometric and trigonometric identities can be proven very easily with algebraic manipulation of dot products.

A pair of vectors is *orthogonal* if their dot product is 0, and the angle between them is 90° (for non-zero vectors). The vector $\bar{0}$ is considered orthogonal to every vector. A *set* of vectors is *orthonormal* if each pair in the set is mutually orthogonal and the norm of each vector is 1. Orthonormal directions are useful because they are employed for transformations of points across different orthogonal coordinate systems with the use of 1-dimensional *projections*. In other words, a new set of coordinates of a data point can be computed with respect to the changed set of directions. This approach is referred to as *coordinate transformation* in analytical geometry, and is also used frequently in linear algebra. The 1-dimensional projection operation of a vector $\bar{x}$ on a unit vector is defined the dot product between the two vectors. It has a natural geometric interpretation as the (positive or negative) distance of $\bar{x}$ from the origin *in the direction of the unit vector*, and therefore it is considered a coordinate in that direction. Consider the point $[10, 15]$ in a 2-dimensional coordinate system. Now imagine that you were given the orthonormal directions $[3/5, 4/5]$ and $[-4/5, 3/5]$. One can represent the point $[10, 15]$ in a new coordinate system defined by the directions $[3/5, 4/5]$ and $[-4/5, 3/5]$ by computing the dot product of $[10, 15]$ with each of these vectors. Therefore, the new coordinates $[x', y']$ are defined as follows:

$$x' = 10 * (3/5) + 15 * (4/5) = 18, \quad y' = 10 * (-4/5) + 15 * (3/5) = 1$$

One can express the original vector using the new axes and coordinates as follows:

$$[10, 15] = x'[3/5, 4/5] + y'[-4/5, 3/5]$$

These types of transformations of vectors to new representations lie at the heart of linear algebra. In many cases, transformed representations of data sets (e.g., replacing each $[x, y]$ in a 2-dimensional data set with $[x', y']$) have useful properties, which are exploited by machine learning applications.

1.2.2 Basic Operations with Vectors and Matrices

The transpose of a matrix is obtained by flipping its rows and columns. In other words, the (i, j) th entry of the transpose is the same as the (j, i) th entry of the original matrix. Therefore, the transpose of an $n \times d$ matrix is a $d \times n$ matrix. The transpose of a matrix A is denoted by A^T . An example of a transposition operation is shown below:

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{31} & a_{32} \end{bmatrix}^T = \begin{bmatrix} a_{11} & a_{21} & a_{31} \\ a_{12} & a_{22} & a_{32} \end{bmatrix}$$

It is easy to see that the transpose of the transpose $(A^T)^T$ of a matrix A is the original matrix A . Like matrices, row vectors can be transposed to column vectors, and vice versa.

Like vectors, matrices can be added only if they have exactly the same sizes. For example, one can add the matrices A and B only if A and B have exactly the same number of rows and columns. The (i, j) th entry of $A+B$ is the sum of the (i, j) th entries of A and B , respectively. The matrix addition operator is commutative, because it inherits the commutative property of scalar addition of its individual entries. Therefore, we have:

$$A + B = B + A$$

A *zero matrix* or *null matrix* is the matrix analog of the scalar value of 0, and it contains only 0s. It is often simply written as “0” even though it is a matrix. It can be added to a matrix of the same size without affecting its values:

$$A + 0 = A$$

Note that matrices, vectors, and scalars all have their own definition of a zero element, which is required to obey the above *additive identity*. For vectors, the zero element is the vector of 0s, and it is written as “ $\vec{0}$ ” with an overbar on top.

It is easy to show that the transpose of the sum of two matrices $A = [a_{ij}]$ and $B = [b_{ij}]$ is given by the sum of their transposes. In other words, we have the following relationship:

$$(A + B)^T = A^T + B^T \quad (1.6)$$

The result can be proven by demonstrating that the (i, j) th element of both sides of the above equation is $(a_{ji} + b_{ji})$.

An $n \times d$ matrix A can either be multiplied with a d -dimensional column vector $\bar{x}$ as $A\bar{x}$, or it can be multiplied with an n -dimensional row vector $\bar{y}$ as $\bar{y}A$. When an $n \times d$ matrix A is multiplied with d -dimensional column vector $\bar{x}$ to create $A\bar{x}$, an element-wise multiplication is performed between the d elements of each row of the matrix A and the d elements of the column vector $\bar{x}$, and then these element-wise products are added to create a scalar. Note that this operation is the same as the dot product, except that one needs to transpose the rows of A to column vectors to rigorously express it as a dot product. This is because dot products are defined between two vectors of the same type (i.e., row vectors or column vectors). At the end of the process, n scalars are computed and arranged into an n -dimensional column vector in which the i th element is the product between the i th row of A and $\bar{x}$. An example of a multiplication of a 3×2 matrix $A = [a_{ij}]$ with a 2-dimensional column vector $\bar{x} = [x_1, x_2]^T$ is shown below:

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{31} & a_{32} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} a_{11}x_1 + a_{12}x_2 \\ a_{21}x_1 + a_{22}x_2 \\ a_{31}x_1 + a_{32}x_2 \end{bmatrix} \quad (1.7)$$

One can also post-multiply an n -dimensional row vector with an $n \times d$ matrix $A = [a_{ij}]$ to create a d -dimensional row vector. An example of the multiplication of a 3-dimensional row vector $\bar{v} = [v_1, v_2, v_3]$ with the 3×2 matrix A is shown below:

$$[v_1, v_2, v_3] \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{31} & a_{32} \end{bmatrix} = [v_1 a_{11} + v_2 a_{21} + v_3 a_{31}, v_1 a_{12} + v_2 a_{22} + v_3 a_{32}] \quad (1.8)$$

It is clear that the multiplication operation between matrices and vectors is not commutative.

The multiplication of an $n \times d$ matrix A with a d -dimensional column vector $\bar{x}$ to create an n -dimensional column vector $A\bar{x}$ is often interpreted as a *linear transformation* from d -dimensional space to n -dimensional space. The precise mathematical definition of a linear transformation is given in Chapter 2. For now, we ask the reader to observe that the result of the multiplication is a weighted sum of the columns of the matrix A , where the weights are provided by the scalar components of vector $\bar{x}$. For example, one can rewrite the matrix-vector multiplication of Equation 1.7 as follows:

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{31} & a_{32} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = x_1 \begin{bmatrix} a_{11} \\ a_{21} \\ a_{31} \end{bmatrix} + x_2 \begin{bmatrix} a_{12} \\ a_{22} \\ a_{32} \end{bmatrix} \quad (1.9)$$

Here, a 2-dimensional vector is mapped into a 3-dimensional vector as a weighted combination of the columns of the matrix. Therefore, the $n \times d$ matrix A is occasionally represented in terms of its ordered set of n -dimensional columns $\bar{a}_1 \dots \bar{a}_d$ as $A = [\bar{a}_1 \dots \bar{a}_d]$. This results in the following form of matrix-vector multiplication using the columns of A and a column vector $\bar{x} = [x_1 \dots x_d]^T$ of coefficients:

$$A\bar{x} = \sum_{i=1}^d x_i \bar{a}_i = \bar{b}$$

Each x_i corresponds to the “weight” of the i th direction $\bar{a}_i$, which is also referred to as the i th *coordinate* of $\bar{b}$ using the (possibly non-orthogonal) directions contained in the columns of A . This notion is a generalization of the (orthogonal) Cartesian coordinates defined by d -dimensional vectors $\bar{e}_1 \dots \bar{e}_d$, where each $\bar{e}_i$ is an axis direction with a single 1 in the i th position and remaining 0s. For the case of the Cartesian system defined by $\bar{e}_1 \dots \bar{e}_d$, the coordinates of $\bar{b} = [b_1 \dots b_d]^T$ are simply $b_1 \dots b_d$, since we have $\bar{b} = \sum_{i=1}^d b_i \bar{e}_i$.

The dot product between two vectors can be viewed as a special case of matrix-vector multiplication. In such a case, a $1 \times d$ matrix (row vector) is multiplied with a $d \times 1$ matrix (column vector), and the result is the same as one would obtain by performing a dot product between the two vectors. However, a subtle difference is that the dot product is defined between two vectors of the same type (typically column vectors) rather than between the matrix representation of a row vector and the matrix representation of a column vector. In order to implement a dot product as a matrix-matrix multiplication, we would first need to convert one of the column vectors into the matrix representation of a row vector, and then perform the matrix multiplication by ordering the “wide” matrix (row vector) before the “tall” matrix (column vector). The resulting 1×1 matrix contains the dot product. For example, consider the dot product in matrix form, which is obtained by matrix-centric multiplication of a row vector with a column vector:

$$\bar{v} \cdot \bar{x} = [v_1, v_2, v_3] \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = [v_1 x_1 + v_2 x_2 + v_3 x_3]$$

The result of the matrix multiplication is a 1×1 matrix containing the dot product, which is a scalar. It is clear that we always obtain the same 1×1 matrix, irrespective of the order of the arguments in the dot product, as long as we transpose the *first* vector in order to place the “wide” matrix before the “tall” matrix:

$$\bar{x} \cdot \bar{v} = \bar{v} \cdot \bar{x}, \quad \bar{x}^T \bar{v} = \bar{v}^T \bar{x}$$

Therefore, dot products are commutative.

However, if we order the “tall” matrix before the “wide” matrix, what we obtain is the *outer product* between the two vectors. The outer product between two 3-dimensional vectors is a 3×3 matrix! In vector form, the outer product is defined between two column vectors $\bar{x}$ and $\bar{v}$ and is denoted by $\bar{x} \otimes \bar{v}$. However, it is easiest to understand the outer product by using the matrix representation of the vectors for multiplication, wherein the first of the vectors is converted into a column vector representation (if needed), and the second of the two vectors is converted into a row vector representation (if needed). In other words, the “tall” matrix is always ordered before the “wide” matrix:

$$\bar{x} \otimes \bar{v} = \bar{x} \bar{v}^T = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} [v_1, v_2, v_3] = \begin{bmatrix} v_1 x_1 & v_2 x_1 & v_3 x_1 \\ v_1 x_2 & v_2 x_2 & v_3 x_2 \\ v_1 x_3 & v_2 x_3 & v_3 x_3 \end{bmatrix}$$

Unlike dot products, outer products can be performed between two vectors of different lengths. Conventionally, outer products are defined between two column vectors, and the second vector is *transposed* into a matrix containing a single row before matrix multiplication. In other words, the j th component of the second vector (in d dimensions) becomes the $(1, j)$ th element of the second matrix (of size $1 \times d$) in the multiplication. The first matrix is simply a $d \times 1$ matrix derived from the column vector. Unlike dot products, the outer product is not commutative; the *order* of the operands matters not only to the values in the final matrix, but also to the size of the final matrix:

$$\bar{x} \otimes \bar{v} \neq \bar{v} \otimes \bar{x}, \quad \bar{x} \bar{v}^T \neq \bar{v} \bar{x}^T$$

The multiplication between vectors, or the multiplication of a matrix with a vector, are both special cases of multiplying two matrices. However, in order to multiply two matrices, certain constraints on their sizes need to be respected. For example, an $n \times k$ matrix U can be multiplied with a $k \times d$ matrix V only because the number of columns k in U is the same as the number of rows k in V . The resulting matrix is of size $n \times d$, in which the (i, j) th entry is the dot product between the vectors corresponding to the i th row of U and the j th column of V . Note that the dot product operations within the multiplication require the underlying vectors to be of the same sizes. The outer product between two vectors is a special case of matrix multiplication that uses $k = 1$ with arbitrary values of n and d ; similarly, the inner product is a special case of matrix multiplication that uses $n = d = 1$, but some arbitrary value of k . Consider the case in which the (i, j) th entries of U and V are u_{ij} and v_{ij} , respectively. Then, the (i, j) th entry of UV is given by the following:

$$(UV)_{ij} = \sum_{r=1}^k u_{ir} v_{rj} \tag{1.10}$$

An example of a matrix multiplication is shown below:

$$\begin{bmatrix} u_{11} & u_{12} \\ u_{21} & u_{22} \\ u_{31} & u_{32} \end{bmatrix} \begin{bmatrix} v_{11} & v_{12} & v_{13} \\ v_{21} & v_{22} & v_{23} \end{bmatrix} = \begin{bmatrix} u_{11}v_{11} + u_{12}v_{21} & u_{11}v_{12} + u_{12}v_{22} & u_{11}v_{13} + u_{12}v_{23} \\ u_{21}v_{11} + u_{22}v_{21} & u_{21}v_{12} + u_{22}v_{22} & u_{21}v_{13} + u_{22}v_{23} \\ u_{31}v_{11} + u_{32}v_{21} & u_{31}v_{12} + u_{32}v_{22} & u_{31}v_{13} + u_{32}v_{23} \end{bmatrix} \quad (1.11)$$

Note that both the two earlier matrix-to-vector and vector-to-matrix multiplications can be viewed as special cases of this more general operation. This is because a d -dimensional row vector can be treated as an $1 \times d$ matrix and a n -dimensional column vector can be treated as a $n \times 1$ matrix. For example, if we multiply this type of special $n \times 1$ matrix with a $1 \times d$ matrix, we will obtain an $n \times d$ matrix with some special properties.

Problem 1.2.2 (Outer Product Properties) *Show that if an $n \times 1$ matrix is multiplied with a $1 \times d$ matrix (which is also an outer product between two vectors), we obtain an $n \times d$ matrix with the following properties: (i) Every row is a multiple of every other row, and (ii) every column is a multiple of every other column.*

It is also possible to show that matrix products can be broken up into the sum of simpler matrices, each of which is an outer product of two vectors. We have already seen that each entry in a matrix product is itself an *inner* product of two vectors extracted from the matrix. What about outer products? It can be shown that the entire matrix is the sum of as many outer products as the common dimension k of the two multiplied matrices:

Lemma 1.2.1 (Matrix Multiplication as Sum of Outer Products) *The product of an $n \times k$ matrix U with a $k \times d$ matrix V results in an $n \times d$ matrix, which can be expressed as the sum of k outer-product matrices; each of these k matrices is the product of an $n \times 1$ matrix with a $1 \times d$ matrix. Each $n \times 1$ matrix corresponds to the i th column U_i of U and each $1 \times d$ matrix corresponds to the i th row V_i of V . Therefore, we have the following:*

$$UV = \sum_{r=1}^k \underbrace{U_r V_r}_{n \times d}$$

Proof: Let u_{ij} and v_{ij} be the (i, j) th entries of U and V , respectively. It can be shown that the r th term in the summation on the right-hand side of the equation in the statement of the lemma contributes $u_{ir}v_{rj}$ to the (i, j) th entry in the summation matrix. Therefore, the overall sum of the terms on the right-hand side is $\sum_{r=1}^k u_{ir}v_{rj}$. This sum is exactly the same as the definition of the (i, j) th term of the matrix multiplication UV (cf. Equation 1.10). ■
In general, matrix multiplication is *not* commutative (except for special cases). In other words, we have $AB \neq BA$ in the general case. This is different from scalar multiplication, which is commutative. A concrete example of non-commutativity is as follows:

$$\begin{bmatrix} 1 & 1 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} 2 & 0 \\ 0 & 0 \end{bmatrix} \neq \begin{bmatrix} 1 & 0 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 0 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 0 & 0 \end{bmatrix}$$

In fact, if the matrices A and B are not square, it might be possible that one of the products, AB , is possible to compute based on the sizes of A and B , whereas BA might not be computable. For example, it is possible to compute AB for the 4×2 matrix A and the 2×5 matrix B . However, it is not possible to compute BA because of mismatching dimensions.

Although matrix multiplication is not commutative, it is associative and distributive:

$$A(BC) = (AB)C, \quad [\text{Associativity}]$$

$$A(B + C) = AB + AC, \quad (B + C)A = BA + CA, \quad [\text{Distributivity}]$$

The basic idea for proving each of the above results is to define variables for the dimensions and entries of each of $A = [a_{ij}]$, $B = [b_{ij}]$, and $C = [c_{ij}]$. Then, an algebraic expression can be computed for the (i, j) th entry on both sides of the equation, and the two are shown to be equal. For example, in the case of associativity, this type of expansion yields the following:

$$[A(BC)]_{ij} = [(AB)C]_{ij} = \sum_k \sum_m a_{ik} b_{km} c_{mj}$$

These properties also hold for matrix-vector multiplication, because all vectors are special cases of matrices. The associativity property is very useful in ensuring efficient matrix multiplication by carefully selecting from the different choices allowed by associativity.

Problem 1.2.3 Express the matrix ABC as the weighted sum of outer products of vectors extracted from A and C . The weights are extracted from matrix B .

Problem 1.2.4 Let A be an 1000000×2 matrix. Suppose you have to compute the 2×1000000 matrix $A^T A A^T$ on a computer with limited memory. Would you prefer to compute $(A^T A)A^T$ or would you prefer to compute $A^T(AA^T)$?

Problem 1.2.5 Let D be an $n \times d$ matrix for which each column sums to 0. Let A be an arbitrary $d \times d$ matrix. Show that the sum of each column of DA is also zero.

The key point in showing the above result is to use the fact that the sum of the rows of D can be expressed as $\bar{e}^T D$, where $\bar{e}$ is a column vector of 1s.

The transpose of the product of two matrices is given by the product of their transposes, but the order of multiplication is reversed:

$$(AB)^T = B^T A^T \quad (1.12)$$

This result can be easily shown by working out the algebraic expression for the (i, j) th entry in terms of the entries of $A = [a_{ij}]$ and $B = [b_{ij}]$. The result for transposes can be easily extended to any number of matrices, as shown below:

Problem 1.2.6 Show the following result for matrices $A_1 \dots A_n$:

$$(A_1 A_2 A_3 \dots A_n)^T = A_n^T A_{n-1}^T \dots A_2^T A_1^T$$

The multiplication between a matrix and a vector also satisfies the same type of transposition rule as shown above.

1.2.3 Special Classes of Matrices

A *symmetric matrix* is a square matrix that is its own transpose. In other words, if A is a symmetric matrix, then we have $A = A^T$. An example of a 3×3 symmetric matrix is shown below:

$$\begin{bmatrix} 2 & 1 & 3 \\ 1 & 4 & 5 \\ 3 & 5 & 6 \end{bmatrix}$$

Note that the (i, j) th entry is always equal to the (j, i) th entry for each $i, j \in \{1, 2, 3\}$.

Problem 1.2.7 If A and B are symmetric matrices, then show that AB is symmetric if and only if $AB = BA$.

The *diagonal* of a matrix is defined as the set of entries for which the row and column indices are the same. Although the notion of diagonal is generally used for square matrices, the definition is sometimes also used for rectangular matrices; in such a case, the diagonal starts at the upper-left corner so that the row and column indices are the same. A square matrix that has values of 1 in all entries along the diagonal and 0s for all non-diagonal entries is referred to as an *identity matrix*, and is denoted by I . In the event that the non-diagonal entries are 0, but the diagonal entries are different from 1, the resulting matrix is referred to as a *diagonal matrix*. Therefore, the identity matrix is a special case of a diagonal matrix. Multiplying an $n \times d$ matrix A with the identity matrix of the appropriate size in any order results in the same matrix A . One can view the identity matrix as the analog of the value of 1 in scalar multiplication:

$$AI = IA = A \quad (1.13)$$

Since A is an $n \times d$ matrix, the size of the identity matrix I in the product AI is $d \times d$, whereas the size of the identity matrix in the product IA is $n \times n$. This is somewhat confusing, because the same notation I in Equation 1.13 refers to identity matrices of two different sizes. In such cases, ambiguity is avoided by subscripting the identity matrix to indicate its size. For example, an identity matrix of size $d \times d$ is denoted by I_d . Therefore, a more unambiguous form of Equation 1.13 is as follows:

$$AI_d = I_n A = A \quad (1.14)$$

Although diagonal matrices are assumed to be square by default, it is also possible to create a relaxed definition¹ of a diagonal matrix, which is not square. In this case, the diagonal is aligned with the *upper-left corner* of the matrix. Such matrices are referred to as *rectangular diagonal matrices*.

Definition 1.2.1 (Rectangular Diagonal Matrix) A rectangular diagonal matrix is an $n \times d$ matrix in which each entry (i, j) has a non-zero value if and only if $i = j$. Therefore, the diagonal of non-zero entries starts at the upper-left corner of the matrix, although it might not meet the lower-right corner.

A *block diagonal matrix* contains square blocks $B_1 \dots B_r$ of (possibly) non-zero entries along the diagonal. All other entries are zero. Although each block is square, they need not be of the same size. Examples of different types of diagonal and block diagonal matrices are shown in the top row of Figure 1.3.

A generalization of the notion of a diagonal matrix is that of a *triangular matrix*:

Definition 1.2.2 (Upper and Lower Triangular Matrix) A square matrix is an **upper triangular matrix** if all entries (i, j) below its main diagonal (i.e., satisfying $i > j$) are zeros. A matrix is **lower triangular** if all entries (i, j) above its main diagonal (i.e., satisfying $i < j$) are zeros.

Definition 1.2.3 (Strictly Triangular Matrix) A matrix is said to be **strictly triangular** if it is triangular and all its diagonal elements are zeros.

¹Instead of referring to such matrices as rectangular diagonal matrices, some authors use a quotation around the word *diagonal*, while referring to such matrices. This is because the word “diagonal” was originally reserved for square matrices.

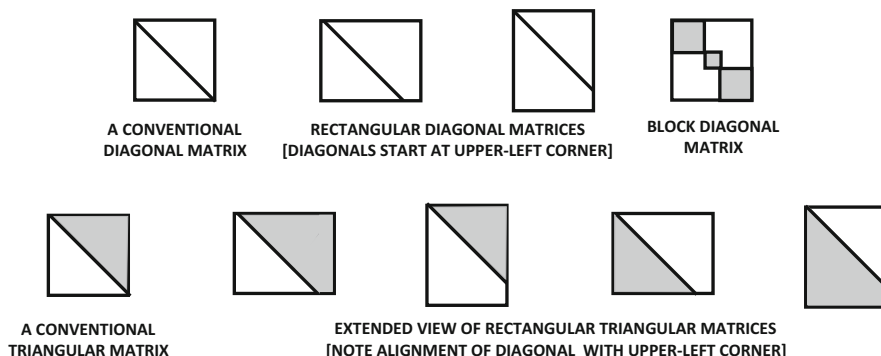


Figure 1.3: Examples of conventional/rectangular diagonal and triangular matrices

We make an important observation about operations on pairs of upper-triangular matrices.

Lemma 1.2.2 (Sum or Product of Upper-Triangular Matrices) *The sum of upper-triangular matrices is upper triangular. The product of upper-triangular matrices is upper triangular.*

Proof Sketch: This result is easy to show by proving that the scalar expressions for the (i, j) th entry in the sum and the product are both 0, when $i > j$. ■

The above lemma naturally applies to lower-triangular matrices as well.

Although the notion of a triangular matrix is generally meant for square matrices, it is sometimes used for rectangular matrices. Examples of different types of triangular matrices are shown in the bottom row of Figure 1.3. The portion of the matrix occupied by non-zero entries is shaded. Note that the number of non-zero entries in rectangular triangular matrices heavily depends on the shape of the matrix. Finally, a matrix A is said to be *sparse*, when most of the entries in it have 0 values. It is often computationally efficient to work with such matrices.

1.2.4 Matrix Powers, Polynomials, and the Inverse

Square matrices can be multiplied with themselves without violating the size constraints of matrix multiplication. Multiplying a square matrix with itself many times is analogous to raising a scalar to a particular power. The n th power of a matrix is defined as follows:

$$A^n = \underbrace{AA \dots A}_{n \text{ times}} \quad (1.15)$$

The zeroth power of a matrix is defined to be the identity matrix of the same size. When a matrix satisfies $A^k = 0$ for some integer k , it is referred to as *nilpotent*. For example, all strictly triangular matrices of size $d \times d$ satisfy $A^d = 0$. Like scalars, one can raise a square matrix to a fractional power, although it is not guaranteed to exist. For example, if $A = V^2$, then we have $V = A^{1/2}$. Unlike scalars, it is not guaranteed that $A^{1/2}$ exists for an arbitrary matrix A , even after allowing for complex-valued entries in the result (see Exercise 14). In general, one can compute a polynomial function $f(A)$ of a square matrix in much the same way as one computes polynomials of scalars. Instead of the constant term used in a scalar polynomial, multiples of the identity matrix are used; the identity matrix

is the matrix analog of the scalar value of 1. For example, the matrix analog of the scalar polynomial $f(x) = 3x^2 + 5x + 2$, when applied to the $d \times d$ matrix A , is as follows:

$$f(A) = 3A^2 + 5A + 2I$$

All polynomials of the same matrix A always commute with respect to the multiplication operator.

Observation 1.2.1 (Commutativity of Matrix Polynomials) *Two polynomials $f(A)$ and $g(A)$ of the same matrix A will always commute:*

$$f(A)g(A) = g(A)f(A)$$

The above result can be shown by expanding the polynomial on both sides, and showing that the same polynomial is reached with the distributive property of matrix multiplication.

Can we raise a matrix to a negative power? The *inverse* of a square matrix A is another square matrix denoted by A^{-1} so that the multiplication of the two matrices (in any order) will result in the identity matrix:

$$AA^{-1} = A^{-1}A = I \quad (1.16)$$

A simple formula exists for inverting 2×2 matrices:

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix}^{-1} = \frac{1}{ad - bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix} \quad (1.17)$$

An example of two matrices that are inverses of each other is shown below:

$$\begin{bmatrix} 8 & 3 \\ 5 & 2 \end{bmatrix} \begin{bmatrix} 2 & -3 \\ -5 & 8 \end{bmatrix} = \begin{bmatrix} 2 & -3 \\ -5 & 8 \end{bmatrix} \begin{bmatrix} 8 & 3 \\ 5 & 2 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

The inverse of a 1×1 matrix containing the element a is simply the 1×1 matrix containing the element $1/a$. Therefore, a matrix inverse naturally generalizes a scalar inverse. Not all matrices have inverses, just as an inverse does not exist for the scalar $a = 0$. A matrix for which an inverse exists is referred to as *invertible* or *nonsingular*. Otherwise, it is said to be *singular*. For example, if the rows in Equation 1.17 are proportional, we would have $ad - bc = 0$, and therefore, the matrix would not be invertible. An example of a matrix that is not invertible is as follows:

$$A = \begin{bmatrix} 1 & 1 \\ 2 & 2 \end{bmatrix}$$

Note that multiplying A with any 2×2 matrix B will always result in a 2×2 matrix AB in which the second row is twice the first. This is not the case for the identity matrix, and, therefore, an inverse of A does not exist. The fact that the rows in the non-invertible matrix A are related by a proportionality factor is not a coincidence. As you will learn in Chapter 2, matrices that are invertible always have the property that a non-zero linear combination of the rows does not sum to zero. In other words, each vector direction in the rows of an invertible matrix must contribute new, non-redundant “information” that cannot be conveyed using sums, multiples, or linear combinations of other directions. The second row of A is twice its first row, and therefore the matrix A is not invertible.

When the inverse of a matrix A does exist, it is unique. Furthermore, the product of a matrix with its inverse is always commutative and leads to the identity matrix. A natural consequence of these facts is that the inverse of the inverse $(A^{-1})^{-1}$ is the original matrix A . We summarize these properties of inverses in the following two lemmas.

Lemma 1.2.3 (Commutativity of Multiplication with Inverse) *If the product AB of $d \times d$ matrices A and B is the identity matrix I , then BA must also be equal to I .*

Proof: We present a restricted proof by making the assumption that a matrix C always exists so that $CA = I$. Then, we have:

$$C = CI = C(AB) = (CA)B = IB = B$$

■

The commutativity of the product of a matrix and its inverse can be viewed as an extension of the statement in Observation 1.2.1 that the product of a matrix A with any polynomial of A is always commutative. A fractional or negative power of a matrix A (like A^{-1}) also commutes with A .

Lemma 1.2.4 *When the inverse of a matrix exists, it is always unique. In other words, if B_1 and B_2 satisfy $AB_1 = AB_2 = I$, we must have $B_1 = B_2$.*

Proof: Since $AB_1 = AB_2$, it follows that $AB_1 - AB_2 = 0$. Therefore, we have $A(B_1 - B_2) = 0$. One can pre-multiply the relationship with B_1 to obtain the following:

$$\underbrace{B_1 A}_I (B_1 - B_2) = 0$$

This proves that $B_1 = B_2$.

■

The negative power A^{-r} for $r > 0$ represents $(A^{-1})^r$. Any polynomial or negative power of a diagonal matrix is another diagonal matrix in which the polynomial function or negative power is applied to each diagonal entry. All diagonal entries of a diagonal matrix need to be non-zero for it to be invertible or have negative powers. The polynomials and inverses of triangular matrices are also triangular matrices of the same type (i.e., lower or upper triangular). A similar result holds for block diagonal matrices.

Problem 1.2.8 (Inverse of Triangular Matrix Is Triangular) *Consider the system of d equations contained in the rows of $R\bar{x} = \bar{e}_k$ for the $d \times d$ upper-triangular matrix R , where $\bar{e}_k$ is a d -dimensional column vector with a single value of 1 in the k th entry and 0 in all other entries. Discuss why solving for $\bar{x} = [x_1 \dots x_d]^T$ is simple in this case by solving for the variables in the order $x_d, x_{d-1}, \dots, x_1$. Furthermore, discuss why the solution for $R\bar{x} = \bar{e}_k$ must satisfy $x_i = 0$ for $i > k$. Why is the solution $\bar{x}$ equal to the k th column of the inverse of R ? Discuss why the inverse of R is also upper-triangular.*

Problem 1.2.9 (Block Diagonal Polynomial and Inverse) *Suppose that you have a block diagonal matrix B , which has blocks $B_1 \dots B_r$ along the diagonal. Show how you can express the polynomial function $f(B)$ and the inverse of B in terms of functions on block matrices.*

The inverse of the product of two square (and invertible) matrices can be computed as a product of their inverses, but with the order of multiplication reversed:

$$(AB)^{-1} = B^{-1}A^{-1} \tag{1.18}$$

Both matrices must be invertible for the product to be invertible. We can use the associativity property of matrix multiplication to show the above result:

$$(AB)(B^{-1}A^{-1}) = A((BB^{-1})A^{-1}) = A((I)A^{-1}) = AA^{-1} = I$$

One can extend the above results to show that $(A_1 A_2 \dots A_k)^{-1} = A_k^{-1} A_{k-1}^{-1} \dots A_1^{-1}$. Note that the individual matrices A_i must be invertible for their product to be invertible. Even if one of the matrices A_i is not invertible, the product will not be invertible (see Exercise 52).

Problem 1.2.10 Suppose that the matrix B is the inverse of matrix A . Show that for any positive integer n , the matrix B^n is the inverse of matrix A^n .

The inversion and the transposition operations can be applied in any order without affecting the result:

$$(A^T)^{-1} = (A^{-1})^T \quad (1.19)$$

This result holds because $A^T(A^{-1})^T = (A^{-1}A)^T = I^T = I$. One can similarly show that $(A^{-1})^T A^T = I$. In other words, $(A^{-1})^T$ is the inverse of A^T .

An *orthogonal matrix* is a square matrix whose inverse is its transpose:

$$AA^T = A^T A = I \quad (1.20)$$

Although such matrices are formally defined in terms of having orthonormal *columns*, the commutativity in the above relationship implies the remarkable property that they contain *both* orthonormal columns and orthonormal rows.

A useful property of invertible matrices is that they define uniquely solvable systems of equations. For example, the solution to $A\bar{x} = \bar{b}$ exists and is uniquely defined as $\bar{x} = A^{-1}\bar{b}$ when A is invertible (cf. Chapter 2). One can also view the solution $\bar{x}$ as a new set of coordinates of $\bar{b}$ in a different (and possibly non-orthogonal) coordinate system defined by the vectors contained in the columns of A . Note that when A is orthogonal, the solution simplifies to $\bar{x} = A^T \bar{b}$, which is equivalent to evaluating the dot product between $\bar{b}$ and each column of A to compute the corresponding coordinate. In other words, we are *projecting* $\bar{b}$ on each orthonormal column of A to compute the corresponding coordinate.

1.2.5 The Matrix Inversion Lemma: Inverting the Sum of Matrices

Is it possible to compute the inverse of the sum of two matrices as a function of polynomials or inverses of the individual matrices? In order to answer this question, note that it is not possible to easily do this even for scalars a and b (which are special cases of matrices). For example, it is not possible to easily express $1/(a+b)$ in terms of $1/a$ and $1/b$. Furthermore, the sum of two matrices A and B need not be invertible even when A and B are invertible. In the scalar case, we might have $a+b=0$, in which case it is not possible to compute $1/(a+b)$. Therefore, it is not easy to compute the inverse of the sum of two matrices.

Some special cases are easier to invert, such as the sum of A with the identity matrix. In such a case, one can generalize the scalar formula for $1/(1+a)$ to matrices. The scalar formula for $1/(1+a)$ for $|a| < 1$ is that of an infinite geometric series:

$$\frac{1}{1+a} = 1 - a + a^2 - a^3 + a^4 + \dots + \text{Infinite Terms} \quad (1.21)$$

The absolute value of a has to be less than 1 for the infinite summation not to blow up. The corresponding analog is the matrix A , which is such that raising it to the n th power causes all the entries of the matrix to go to 0 as $n \Rightarrow \infty$. In other words, the *limit* of A^n as $n \Rightarrow \infty$ is the zero matrix. For such matrices, the following result holds:

$$\begin{aligned} (I + A)^{-1} &= I - A + A^2 - A^3 + A^4 + \dots + \text{Infinite Terms} \\ (I - A)^{-1} &= I + A + A^2 + A^3 + A^4 + \dots + \text{Infinite Terms} \end{aligned}$$

The result can be used for inverting triangular matrices (although more straightforward alternatives exist):

Problem 1.2.11 (Inverting Triangular Matrices) *A $d \times d$ triangular matrix L with non-zero diagonal entries can be expressed in the form $(\Delta + A)$, where Δ is an invertible diagonal matrix and A is a **strictly** triangular matrix. Show how to compute the inverse of L using only diagonal matrix inversions and matrix multiplications/additions. Note that strictly triangular matrices of size $d \times d$ are always nilpotent and satisfy $A^d = 0$.*

It is also possible to derive an expression for inverting the sum of two matrices in terms of the original matrices under the condition that one of the two matrices is “compact.” By compactness, we mean that one of the two matrices has so much structure to it that it can be expressed as the product of two much smaller matrices. The *matrix-inversion lemma* is a useful property for computing the inverse of a matrix after incrementally updating it with a matrix created from the outer-product of two vectors. These types of inverses arise often in iterative optimization algorithms such as the *quasi-Newton method* and for incremental linear regression. In these cases, the inverse of the original matrix is already available, and one can cheaply update the inverse with the matrix inversion lemma.

Lemma 1.2.5 (Matrix Inversion Lemma) *Let A be an invertible $d \times d$ matrix, and $\bar{u}$ and $\bar{v}$ be non-zero d -dimensional column vectors. Then, $A + \bar{u}\bar{v}^T$ is invertible if and only if $\bar{v}^T A^{-1} \bar{u} \neq -1$. In such a case, the inverse is computed as follows:*

$$(A + \bar{u}\bar{v}^T)^{-1} = A^{-1} - \frac{A^{-1}\bar{u}\bar{v}^T A^{-1}}{1 + \bar{v}^T A^{-1} \bar{u}}$$

Proof: If the matrix $(A + \bar{u}\bar{v}^T)$ is invertible, then the product of $(A + \bar{u}\bar{v}^T)$ and A^{-1} is invertible as well (as the product of two invertible matrices). Post-multiplying $(A + \bar{u}\bar{v}^T)A^{-1}$ with $\bar{u}$ yields a non-zero vector, because of the invertibility of the former matrix. Otherwise, we can further pre-multiply the resulting equation $(A + \bar{u}\bar{v}^T)A^{-1}\bar{u} = 0$ with the inverse of $(A + \bar{u}\bar{v}^T)A^{-1}$ in order to yield $\bar{u} = \bar{0}$, which is against the assumptions of the lemma. Therefore, we have:

$$\begin{aligned} (A + \bar{u}\bar{v}^T)A^{-1}\bar{u} &\neq 0 \\ \bar{u} + \bar{u}\bar{v}^T A^{-1}\bar{u} &\neq 0 \\ \bar{u}(1 + \bar{v}^T A^{-1}\bar{u}) &\neq 0 \\ 1 + \bar{v}^T A^{-1}\bar{u} &\neq 0 \end{aligned}$$

Therefore, the precondition of invertibility is shown.

Conversely, if the precondition $1 + \bar{v}^T A^{-1}\bar{u} \neq 0$ holds, we can show that the matrix $P = A^{-1} - \frac{A^{-1}\bar{u}\bar{v}^T A^{-1}}{1 + \bar{v}^T A^{-1}\bar{u}}$ is a valid inverse of $Q = (A + \bar{u}\bar{v}^T)$. Note that the matrix P is well defined only when the precondition holds. In such a case, expanding both PQ and QP algebraically yields the identity matrix. For example, expanding PQ yields the following:

$$\begin{aligned} PQ &= I + A^{-1}\bar{u}\bar{v}^T - \frac{A^{-1}\bar{u}\bar{v}^T + A^{-1}\bar{u}[\bar{v}^T A^{-1}\bar{u}]\bar{v}^T}{1 + \bar{v}^T A^{-1}\bar{u}} \\ &= I + A^{-1}\bar{u}\bar{v}^T - \frac{A^{-1}\bar{u}\bar{v}^T(1 + [\bar{v}^T A^{-1}\bar{u}])}{1 + \bar{v}^T A^{-1}\bar{u}} \\ &= I + A^{-1}\bar{u}\bar{v}^T - A^{-1}\bar{u}\bar{v}^T = I \end{aligned}$$

Although matrix multiplication is not commutative in general, the above proof uses the fact that the scalar $\bar{v}^T A^{-1} \bar{u}$ can be moved around in the order of matrix multiplication because it is a scalar. ■

Variants of the matrix inversion lemma are used in various types of iterative updates in machine learning. A specific example is *incremental* linear regression, where one often wants to invert matrices of the form $C = D^T D$, where D is an $n \times d$ data matrix. When a new d -dimensional data point $\bar{v}$ is received, the size of the data matrix becomes $(n+1) \times d$ with the addition of row vector $\bar{v}^T$ to D . The matrix C is now updated to $D^T D + \bar{v} \bar{v}^T$, and the matrix inversion lemma comes in handy for updating the inverted matrix in $O(d^2)$ time. One can even generalize the above result to cases where the vectors $\bar{u}$ and $\bar{v}$ are replaced with “thin” matrices U and V containing a small number k of columns.

Theorem 1.2.1 (Sherman–Morrison–Woodbury Identity) *Let A be an invertible $d \times d$ matrix and let U, V be $d \times k$ non-zero matrices for some small value of k . Then, the matrix $A + UV^T$ is invertible if and only if the $k \times k$ matrix $(I + V^T A^{-1} U)$ is invertible. Furthermore, the inverse is given by the following:*

$$(A + UV^T)^{-1} = A^{-1} - A^{-1}U(I + V^T A^{-1}U)^{-1}V^T A^{-1}$$

This type of update is referred to as a *low-rank update*; the notion of rank will be explained in Chapter 2. We provide some exercises relevant to the matrix inversion lemma.

Problem 1.2.12 *Suppose that I and P are two $k \times k$ matrices. Show the following result:*

$$(I + P)^{-1} = I - (I + P)^{-1}P$$

A hint for solving this problem is to check what you get when you left multiply both sides of the above identity with $(I + P)$. A closely related result is the *push-through identity*:

Problem 1.2.13 (Push-Through Identity) *If U and V are two $n \times d$ matrices, show the following result:*

$$U^T(I_n + VU^T)^{-1} = (I_d + U^T V)^{-1}U^T$$

Use the above result to show the following for any $n \times d$ matrix D and scalar $\lambda > 0$:

$$D^T(\lambda I_n + DD^T)^{-1} = (\lambda I_d + D^T D)^{-1}D^T$$

A hint for solving the above problem is to see what happens when one left-multiplies *and* right-multiplies the above identities with the appropriate matrices. The push-through identity derives its name from the fact that we push in a matrix on the left and it comes out on the right. This identity is very important and is used repeatedly in this book.

1.2.6 Frobenius Norm, Trace, and Energy

Like vectors, one can define norms of matrices. For the rectangular $n \times d$ matrix A with (i, j) th entry denoted by a_{ij} , its *Frobenius norm* is defined as follows:

$$\|A\|_F = \|A^T\|_F = \sqrt{\sum_{i=1}^n \sum_{j=1}^d a_{ij}^2} \quad (1.22)$$

Note the use of $\|\cdot\|_F$ to denote the Frobenius norm. The squared Frobenius norm is the sum of squares of the norms of the row-vectors (or, alternatively, column vectors) in the

matrix. It is invariant to matrix transposition. The *energy* of a matrix A is an alternative term used in machine learning community for the squared Frobenius norm.

The *trace* of a square matrix A , denoted by $\text{tr}(A)$, is defined by the sum of its diagonal entries. The energy of a rectangular matrix A is equal to the trace of either AA^T or $A^T A$:

$$\|A\|_F^2 = \text{Energy}(A) = \text{tr}(AA^T) = \text{tr}(A^T A) \quad (1.23)$$

More generally, the trace of the product of two matrices $C = [c_{ij}]$ and $D = [d_{ij}]$ of sizes of $n \times d$ is the sum of their entrywise product:

$$\text{tr}(CD^T) = \text{tr}(DC^T) = \sum_{i=1}^n \sum_{j=1}^d c_{ij} d_{ij} \quad (1.24)$$

The trace of the product of two matrices $A = [a_{ij}]_{n \times d}$ and $B = [b_{ij}]_{d \times n}$ is invariant to the order of matrix multiplication:

$$\text{tr}(AB) = \text{tr}(BA) = \sum_{i=1}^n \sum_{j=1}^d a_{ij} b_{ji} \quad (1.25)$$

Problem 1.2.14 *Show that the Frobenius norm of the outer product of two vectors is equal to the product of their Euclidean norms.*

The Frobenius norm shares many properties with vector norms, such as *sub-additivity* and *sub-multiplicativity*. These properties are analogous to the triangle inequality and the Cauchy-Schwarz inequality, respectively, in the case of vector norms.

Lemma 1.2.6 (Sub-additive Frobenius Norm) *For any pair of matrices A and B of the same size, the triangle inequality $\|A + B\|_F \leq \|A\|_F + \|B\|_F$ is satisfied.*

The above result is easy to show by simply treating a matrix as a vector and creating two long vectors from A and B , each with dimensionality equal to the number of matrix entries.

Lemma 1.2.7 (Sub-multiplicative Frobenius Norm) *For any pair of matrices A and B of sizes $n \times k$ and $k \times d$, respectively, the sub-multiplicative property $\|AB\|_F \leq \|A\|_F \|B\|_F$ is satisfied.*

Proof Sketch: Let $\bar{a}_1 \dots \bar{a}_n$ correspond to the rows of A , and $\bar{b}_1 \dots \bar{b}_d$ contain the transposed columns of B . Then, the (i, j) th entry of AB is $\bar{a}_i \cdot \bar{b}_j$, and the squared Frobenius norm of the matrix AB is $\sum_{i=1}^n \sum_{j=1}^d (\bar{a}_i \cdot \bar{b}_j)^2$. Each $(\bar{a}_i \cdot \bar{b}_j)^2$ is less than $\|\bar{a}_i\|^2 \|\bar{b}_j\|^2$ according to the Cauchy-Schwarz inequality. Therefore, we have the following:

$$\|AB\|_F^2 = \sum_{i=1}^n \sum_{j=1}^d (\bar{a}_i \cdot \bar{b}_j)^2 \leq \sum_{i=1}^n \sum_{j=1}^d \|\bar{a}_i\|^2 \|\bar{b}_j\|^2 = \left(\sum_{i=1}^n \|\bar{a}_i\|^2 \right) \left(\sum_{j=1}^d \|\bar{b}_j\|^2 \right) = \|A\|_F^2 \|B\|_F^2$$

Computing the square-root of both sides yields the desired result. ■

Problem 1.2.15 (Small Matrices Have Large Inverses) *Show that the Frobenius norm of the inverse of an $n \times n$ matrix with Frobenius norm of ϵ is at least $\sqrt{n}/\epsilon$.*

1.3 Matrix Multiplication as a Decomposable Operator

Matrix multiplication can be viewed as a vector-to-vector function that maps one vector to another. For example, the multiplication of a d -dimensional column vector $\bar{x}$ with the $d \times d$ matrix A maps it to another d -dimensional vector, which is the output of the function $f(\bar{x})$:

$$f(\bar{x}) = A\bar{x}$$

One can view this function as a vector-centric generalization of the univariate linear function $g(x) = ax$ for scalar a . This is one of the reasons that matrices are viewed as *linear operators* on vectors. Much of linear algebra is devoted to understanding this transformation and leveraging it for efficient numerical computations.

One issue is that if we have a large $d \times d$ matrix, it is often hard to interpret what the matrix is really doing to the vector in terms of its individual components. This is the reason that it is often useful to interpret a matrix as a product of simpler matrices. Because of the beautiful property of the associativity of matrix multiplication, one can interpret a product of simple matrices (and a vector) as the *composition of simple operations on the vector*. In order to understand this point, consider the case when the above matrix A can be decomposed into the product of simpler $d \times d$ matrices $B_1, B_2, \dots, B_k$, as follows:

$$A = B_1 B_2 \dots B_{k-1} B_k$$

Assume that each B_i is simple enough that one can intuitively interpret the effect of multiplying a vector $\bar{x}$ with B_i easily (such as rotating the vector or scaling it). Then, the aforementioned function $f(\bar{x})$ can be written as follows:

$$\begin{aligned} f(\bar{x}) &= A\bar{x} = [B_1 B_2 \dots B_{k-1} B_k] \bar{x} \\ &= B_1 (B_2 \dots [B_{k-1} (B_k \bar{x})]) \quad [\text{Associative Property of Matrix Multiplication}] \end{aligned}$$

The nested brackets on the right provide an order to the operations. In other words, we first apply the operator B_k to $\bar{x}$, then apply B_{k-1} , and so on all the way down to B_1 . Therefore, *as long as we can decompose a matrix into the product of simpler matrices, we can interpret matrix multiplication with a vector as a sequence of simple, easy-to-understand operations on the vector*. In this section, we will provide two important examples of decomposition, which will be studied in greater detail throughout the book.

1.3.1 Matrix Multiplication as Decomposable Row and Column Operators

An important property of matrix multiplication is that the rows and columns of the product can be manipulated by applying the corresponding operations on one of the two matrices. In a product AX of two matrices A and X , interchanging the i th and j th *rows* of the *first* matrix A will also interchange the corresponding rows in the product (which has the same number of rows as the first matrix). Similarly, if we interchange the *columns* of the *second* matrix, this interchange will also occur in the product (which has the same number of columns as the second matrix). There are three main elementary operations, corresponding to interchange, addition, and multiplication. The elementary row operations on matrices are defined as follows:

- *Interchange operation:* The i th and j th rows of the matrix are interchanged. The operation is fully defined by two indices i and j in any order.

- *Addition operation:* A scalar multiple of the j th row is added to the i th row. The operation is defined by two indices i, j in a specific order, and a scalar multiple c .
- *Scaling operation:* The i th row is multiplied with scalar c . The operation is fully defined by the row index i and the scalar c .

The above operations are referred to as *elementary row operations*. One can define exactly analogous operations on the columns with *elementary column operations*.

An *elementary matrix* is a matrix that differs from the identity matrix by applying a single row or column operation. Pre-multiplying a matrix X with an elementary matrix corresponding to an interchange results in an interchange of the rows of X . In other words, if E is the elementary matrix corresponding to an interchange, then a pair of rows of $X' = EX$ will be interchanged with respect to X . A similar result holds true for other operations like row addition and row scaling. Some examples of 3×3 elementary matrices with the corresponding operations are illustrated in the table below:

Interchange	Addition	Scaling
$\begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$\begin{bmatrix} 1 & c & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$\begin{bmatrix} 1 & 0 & 0 \\ 0 & c & 0 \\ 0 & 0 & 1 \end{bmatrix}$
(a) Interchange rows 1, 2	(b) Add $c \times$ (row 2) to row 1	(c) Multiply row 2 by c

These matrices are also referred to as *elementary matrix operators* because they are used to apply specific row operations on arbitrary matrices. The scalar c is always non-zero in the above matrices, because all elementary matrices are invertible and are different from the identity matrix (albeit in a minor way). Pre-multiplication of X with the appropriate elementary matrix can result in a row exchange, addition, or row-wise scaling being applied to X . For example, the first and second rows of the matrix X can be exchanged to create X' as follows:

$$\underbrace{\begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}}_{\text{Operator}} \underbrace{\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}}_X = \underbrace{\begin{bmatrix} 4 & 5 & 6 \\ 1 & 2 & 3 \\ 7 & 8 & 9 \end{bmatrix}}_{X'}$$

The first row of the matrix can be scaled up by 2 with the use of the appropriate scaling operator:

$$\underbrace{\begin{bmatrix} 2 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}}_{\text{Operator}} \underbrace{\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}}_X = \underbrace{\begin{bmatrix} 2 & 4 & 6 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}}_{X'}$$

Post-multiplication of matrix X with the following elementary matrices will result in exactly analogous operations on the columns of X to create X' :

Interchange	Addition	Scaling
$\begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$\begin{bmatrix} 1 & 0 & 0 \\ c & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$\begin{bmatrix} 1 & 0 & 0 \\ 0 & c & 0 \\ 0 & 0 & 1 \end{bmatrix}$
(a) Interchange col. 1, 2	(b) Add $c \times$ (col. 2) to col. 1	(c) Multiply col. 2 by c

Only the elementary matrix for the addition operation is slightly different between row and column operations (although the other two matrices are the same). In the following, we show an example of how post-multiplication with the appropriate elementary matrix can result in a column exchange operation:

$$\underbrace{\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}}_X \underbrace{\begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}}_{\text{Operator}} = \underbrace{\begin{bmatrix} 2 & 1 & 3 \\ 5 & 4 & 6 \\ 8 & 7 & 9 \end{bmatrix}}_{X'}$$

Note that this example is very similar to the one provided for row interchange, except that the corresponding elementary matrix is *post-multiplied* in this case.

Problem 1.3.1 Define a 4×4 operator matrix so that pre-multiplying any matrix X with this matrix will result in addition of c_i times the i th row of X to the 2nd row of X for each $i \in \{1, 2, 3, 4\}$ in one shot. Show that this matrix can be expressed as the product of three elementary addition matrices and a single elementary multiplication matrix.

These types of elementary matrices are always invertible. The inverse of the interchange matrix is itself. The inverse of the scaling matrix is obtained by replacing the entry c with $1/c$. The inverse of the row or column addition matrix is obtained by replacing c with $-c$. We make the following observation:

Observation 1.3.1 The inverse of an elementary matrix is another elementary matrix.

Keeping the inverses of elementary matrices in mind can sometimes be useful. Therefore, the reader is encouraged to work out the details of these matrices using the exercise below:

Problem 1.3.2 Write down one example of each of the three types [i.e., interchange, multiplication, and addition] of elementary matrices for performing row operations on a matrix of size 4×4 . Work out the inverse of these matrices. Repeat this result for each of the three types of matrices for performing column operations.

The following exercises are examples of the utility of the inverses of elementary matrices:

Problem 1.3.3 Let A and B be two matrices. Let A_{ij} be the matrix obtained by exchanging the i th and j th columns of A , and B_{ij} be the matrix obtained by exchanging the i th and j th rows of B . Write each of A_{ij} and B_{ij} in terms of A or B , and an elementary matrix. Now explain why $A_{ij}B_{ij} = AB$.

Problem 1.3.4 Let A and B be two matrices. Let matrix A' be created by adding c times the j th column of A to its i th column, and matrix B' be created by subtracting c times the i th row of B from its j th row. Explain using the concept of elementary matrices why the matrices AB and $A'B'$ are the same.

It is also possible to apply elementary operations to matrices that are not square. For an $n \times d$ matrix, the pre-multiplication operator matrix will be of size $n \times n$, whereas the post-multiplication operator matrix will be of size $d \times d$.

Permutation Matrices

An elementary row (or column) interchange operator matrix is a special case of a *permutation matrix*. A permutation matrix contains a single 1 in each row, and a single 1 in each column. An example of a permutation matrix P is shown below:

$$P = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

Pre-multiplying any matrix with a permutation matrix shuffles the rows, and post-multiplying any matrix with a permutation matrix shuffles the columns. For example, pre-multiplying any four-row matrix with the above matrix P reorders the rows as follows:

$$\text{Row 3} \Rightarrow \text{Row 1} \Rightarrow \text{Row 4} \Rightarrow \text{Row 2}$$

Post-multiplying any four-column matrix with P reorders the columns, albeit in the reverse order:

$$\text{Column 2} \Rightarrow \text{Column 4} \Rightarrow \text{Column 1} \Rightarrow \text{Column 3}$$

It is noteworthy that a permutation matrix and its transpose are inverses of one another because they have orthonormal columns. Such matrices are useful in reordering the items of a data matrix, and applications will be shown for graph matrices in Chapter 10. Since one can shuffle the rows of a matrix by using a sequence of row interchange operations, it follows that *any permutation matrix is a product of row interchange operator matrices*.

Applications of Elementary Operator Matrices

The row manipulation property is used to compute the inverses of matrices. This is because a matrix A and its inverse X are related as follows:

$$AX = I$$

Row operations are applied on A to convert the matrix to the identity matrix. A systematic approach to perform such row operations to convert A to the identity matrix is the *Gaussian elimination method* discussed in Chapter 2. These operations are mirrored on the right-hand side so that the identity matrix is converted to the inverse. As the final result of the row operations, we obtain the following:

$$IX = A^{-1}$$

Elementary matrices are fundamental because *one can decompose any square and invertible matrix into a product of elementary matrices*. In fact, if one is willing to augment the set of elementary multiplication operators to allow the scalar c on the diagonal to be zero (which is traditionally not the case), then one can express any square matrix as a product of augmented elementary matrices.

Finally, we discuss the important application of finding a solution to the system of equations $A\bar{x} = \bar{b}$. Here, A is an $n \times d$ matrix, $\bar{x}$ is d -dimensional column vector, and $\bar{b}$ is an n -dimensional row vector. Note that a feasible solution might not exist to this system of equations, especially when some groups of equations are mutually inconsistent. For example, the equations $\sum_{i=1}^{100} x_i = +1$ and $\sum_{i=1}^{100} x_i = -1$ are mutually inconsistent.

The matrix-centric methodology for solving such a system of linear equations derives its inspiration from the well-known methodology of eliminating variables from systems of equations in multiple variables. For example, if we have a pair of linear equations in x_1 and x_2 , we can create an equation without one of the variables by subtracting an appropriate multiple of one equation from the other. This operation is identical to the elementary row addition operation discussed in this chapter. This general principle can be applied to systems containing any number of variables, so that the r th equation is defined only in terms of $x_r, x_{r+1}, \dots, x_d$. This is equivalent to converting the original system $A\bar{x} = \bar{b}$ into a new system $A'\bar{x} = \bar{b}'$ where A' is triangular. Therefore, if we apply a sequence $E_1 \dots E_k$ of elementary row operations to the system of equations, we obtain the following relationship:

$$\underbrace{E_k E_{k-1} \dots E_1 A}_{A'} \bar{x} = \underbrace{E_k E_{k-1} \dots E_1 \bar{b}}_{\bar{b}'}$$

A triangular system of equations is solved by first processing equations with fewer variables and iteratively backsubstituting these values to reduce the system to fewer variables. These methods will be discussed in detail in Chapter 2. It is noteworthy that the problem of solving linear equations is a special case of the fundamental machine learning problem of *linear regression*, in which the best-fit solution is found to an inconsistent system of equations. Linear regression serves as the “parent problem” to many machine learning problems like *least-squares classification*, *support-vector machines*, and *logistic regression*.

1.3.2 Matrix Multiplication as Decomposable Geometric Operators

Aside from decompositions involving elementary matrices, other forms of decompositions are based on matrices with *geometric* interpretations, such as rotation, reflection, and scaling. For example, a 90° counter-clockwise rotation of the vector $[2, 1]$ transforms it to $[-1, 2]$. A reflection of the point $[2, 1]$ across the X -axis yields $[2, -1]$; a scaling along the X -axis and Y -axis by respective factors of 2 and 3 yields $[4, 3]$. All these simple transformations on a vector in two dimensions can be defined by pre-multiplication of the corresponding column vector with a 2×2 matrix (or post-multiplication of a row vector with the transpose of this 2×2 matrix). For example, consider the column vector representation of a point with polar coordinates $[a, \alpha]$ and Cartesian coordinates $[a \cos(\alpha), a \sin(\alpha)]$. The point has magnitude a and makes a counter-clockwise angle of α with the X -axis. Then, one can multiply it with the *rotation matrix* shown below to yield a counter-clockwise rotation of the vector with angle θ :

$$\begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix} \begin{bmatrix} a \cos(\alpha) \\ a \sin(\alpha) \end{bmatrix} = \begin{bmatrix} a[\cos(\alpha)\cos(\theta) - \sin(\alpha)\sin(\theta)] \\ a[\cos(\alpha)\sin(\theta) + \sin(\alpha)\cos(\theta)] \end{bmatrix} = \begin{bmatrix} a \cos(\alpha + \theta) \\ a \sin(\alpha + \theta) \end{bmatrix}$$

The final result is obtained by using a standard trigonometric identity for the cosines and sines of the sums of angles, and the Cartesian coordinates shown on the right-hand side are equivalent to the polar coordinates $[a, \alpha + \theta]$. In other words, the original coordinates $[a, \alpha]$ have been rotated counter-clockwise by angle θ . The basic geometric operations like rotation, reflection, and scaling can be performed by post-multiplication with appropriately chosen matrices. We list these matrices below, which are defined for pre-multiplying column vectors:

Rotation	Reflection	Scaling
$\begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}$	$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$	$\begin{bmatrix} c_1 & 0 \\ 0 & c_2 \end{bmatrix}$
(a) Rotate counter-clockwise by θ	(b) Reflect across X -axis	(c) Scale x and y by factors of c_1 and c_2

The above matrices are also referred to as *elementary* matrices for geometric operations (like the elementary matrices for row and column operations). It is possible for the diagonal entries of the scaling matrix to be negative or 0. Strictly speaking, the elementary reflection matrix can be considered a special case of the scaling matrix by setting the different values of c_i to values drawn from $\{-1, 1\}$.

Problem 1.3.5 *The above list of matrices for rotation, reflection, and scaling is designed to transform a column vector $\bar{x}$ using the matrix-to-vector product $A\bar{x}$. Write down the corresponding matrices for the case when you want to transform a row vector $\bar{u}$ as $\bar{u}B$.*

The matrix for a sequence of transformations can be computed by multiplying the corresponding matrices. This is easy to show by observing that if we have $A = A_1 \dots A_k$, then successively pre-multiplying a column-vector $\bar{x}$ with $A_k \dots A_1$ is the same as the expression $A_1(A_2(\dots(A_k\bar{x})))$. Because of the associativity of matrix multiplication, one can express this matrix as $(A_1 \dots A_k)\bar{x} = A\bar{x}$. Conversely, if a matrix can be expressed as a product of simpler matrices (like the geometric ones shown above), then multiplication of a vector with that matrix is equivalent to a sequence of the above geometric transformations.

A fundamental result of linear algebra is that any square matrix can be shown to be a product of rotation/reflection/scaling matrices by using a technique called *singular value decomposition*. In other words, *all linear transformations of vectors defined by matrix multiplication corresponding to the application of a sequence of rotations, reflections, and scaling on the vector*. Chapter 2 generalizes the 2×2 matrices in the above table to any number of dimensions by using $d \times d$ matrices. These concepts are sometimes more complex in higher dimensions — for example, it is possible to use an arbitrarily oriented axis of rotation in higher dimensions unlike in the case of two dimensions. The decomposition of a matrix into geometrically interpretable matrices can also be used for computing inverses.

Problem 1.3.6 *Suppose that you are told that any invertible square matrix A can be expressed as a product of elementary rotation/reflection/scaling matrices as $A = R_1 R_2 \dots R_k$. Express the inverse of A in terms of the easily computable inverses of $R_1, R_2, \dots, R_k$.*

It is also helpful to understand the row addition operator, discussed in the previous section. Consider the 2×2 row-addition operator:

$$A = \begin{bmatrix} 1 & c \\ 0 & 1 \end{bmatrix}$$

This operator *shears* the space along the direction of the first coordinate. For example, if vector $\bar{z}$ is $[x, y]^T$, then $A\bar{z}$ yields the new vector $[x + cy, y]^T$. Here, the y -coordinate remains unchanged, whereas the x -coordinate gets sheared in proportion to its height. The shearing of a rectangle into a parallelogram is shown in Figure 1.4. An elementary row operator matrix is a very special case of a triangular matrix; correspondingly, a triangular matrix *with unit diagonal entries* corresponds to a sequence of shears. This is because one can convert an identity matrix into any such triangular matrix with a sequence of elementary row addition operations.

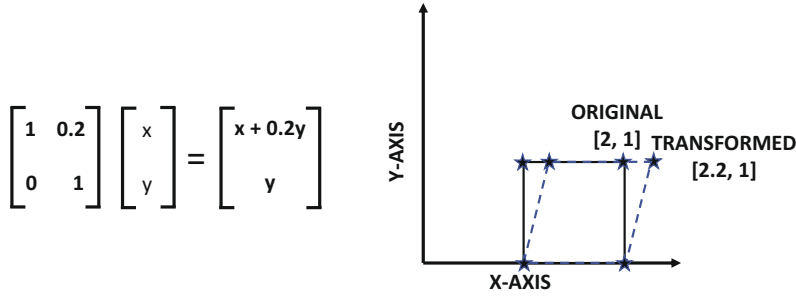


Figure 1.4: An elementary row addition operator can be interpreted as a shear transform

As discussed earlier in this section, a linear transformation can be viewed as a succession of simpler transforms. This simpler sequence of transforms is obtained by *decomposing* a matrix A into the product of simpler matrices $B_1 \dots B_k$ as follows:

$$f(\bar{x}) = A\bar{x} = B_1(B_2 \dots [B_{k-1}(B_k\bar{x})])$$

Each B_i is typically a group of similar transforms, such as orthogonal matrices (sequence of rotations), diagonal matrices (sequence of scalings), or triangular matrices with unit diagonal entries (sequence of shears). There is considerable flexibility in terms of how these decompositions can be performed. For example, this book discusses the *LU decomposition*, the *QR decomposition*, and the *singular value decomposition*.

1.4 Basic Problems in Machine Learning

Machine learning is about constructing models on observed examples in the rows of data matrices, and using these models to make predictions about missing entries of previously unseen examples. This process is also referred to as *learning*, which is where “machine learning” derives its name. Throughout this book, we assume that we have an $n \times d$ data matrix D , which contains n examples of d -dimensional data points in its rows. A *dimension* or *attribute* is one of the d properties of a data point, and a column of D contains this property for all data instances. For example, in a medical application, each row of the data matrix D might correspond to a patient, and the d dimensions might represent the different attributes garnered from the patient, such as their height, weight, test results, and so on. Machine learning uses these examples for various applications, such as that of predicting the value of a particular dimension in the data, finding anomalous patients, or grouping similar patients. These correspond to classical problems in machine learning, such as classification, anomaly detection, and clustering. This section will introduce these classical problems.

1.4.1 Matrix Factorization

Matrix factorization is an alternative term for matrix decomposition, and it usually refers to an optimization-centric view of decomposition. Matrix factorization decomposes an $n \times d$ matrix D into two *factor* matrices U and V of respective sizes $n \times k$ and $d \times k$, so that $UV^T \approx D$. Here, $k \ll \min\{n, d\}$ is a parameter referred to as the *rank* of the factorization. The notion of rank is introduced formally in Chapter 2. The rank controls the “conciseness” of the factorization because the total number of entries in U and V is $k(n+d)$, which is much

smaller than the original number of entries in D . Matrix factorization is a generalization of (real-valued) scalar factorization to matrices. There are an infinite number of factors of the same matrix D , just as a scalar can be factored in an infinite number of pairs of real values. For example, the scalar 6 can be written as 2×3 , as 1.5×4 , or as $\sqrt{2} \times \sqrt{18}$. An example of a matrix factorization of a 3×3 matrix into two smaller matrices is shown below:

$$\begin{bmatrix} 1 & -1 & 1 \\ -1 & 1 & -1 \\ 2 & -2 & 2 \end{bmatrix} = \begin{bmatrix} 1 \\ -1 \\ 2 \end{bmatrix} [1, -1, 1]$$

In the above case, the factorization is exact, although it is often allowed to be *approximately* true in order to minimize the sizes of the factor matrices U and V . If one is willing to allow for a reasonable level of approximation, the value of k can be quite small.

A common approach for matrix factorization is to set up the following optimization problem:

$$\text{Minimize } J = \|D - UV^T\|_F^2 \quad (1.26)$$

Here, $\|\cdot\|_F^2$ refers to the squared Frobenius norm, which is the sum of the squares of the entries in the *residual* matrix $(D - UV^T)$. The objective function J is minimized with the use of gradient descent on the parameter matrices U and V , whose entries are variables of this optimization problem. By minimizing this objective function, one will ensure that the matrix $(D - UV^T)$ will have entries that are small in magnitude, and therefore $D \approx UV^T$. These types of objective functions are also referred to as *loss functions*, because they measure how much information UV^T “loses” with respect to the original matrix D .

One can even factorize an incompletely specified matrix D by formulating the optimization objective function only with the observed entries. This basic principle serves as the foundation of *recommender systems*. For example, consider a setting in which we have n users and d ratings; the (i, j) th entry of D provides the rating of the user i for item j . Most of the entries of D are unobserved, because users typically rate only a small subset of items. In such a case, the objective function $\|D - UV^T\|_F^2$ will need to be modified, so that we sum up the squared errors only over the *observed* entries in D . This is because the values of the remaining entries in $(D - UV^T)$ are unknown. Setting up an optimization problem only in terms of a subset of entries allows us to learn fully specified matrices U and V . Therefore, UV^T provides a prediction of the *fully reconstructed* matrix D . This application will be discussed in greater detail in Chapter 8.

1.4.2 Clustering

The problem of clustering is that of partitioning the rows of the $n \times d$ data matrix D into groups of similar rows. For example, imagine a setting where one has data records in which the rows of D correspond to different individuals, and the different dimensions (columns) of D correspond to the number of units of each product bought in a supermarket. Then, a clustering application might try to segment the data set into groups of similar individuals with particular types of buying behavior. The number of clusters might either be specified by the analyst up front, or the algorithm might use a heuristic to set the number of “natural” clusters in the data. One can often use the segmentation created by clustering as a preprocessing step for other analytical goals. For example, on closer examination of the clusters, one might learn that particular individuals are interested in household articles in a grocery store, whereas others are interested in fruits. This information can be used by the supermarket to make recommendations. Various clustering algorithms like *k-means* and *spectral clustering* are introduced in Chapters 8, 9, and 10.

1.4.3 Classification and Regression Modeling

The problem of classification is closely related to clustering, except that more guidance is available for grouping the data with the use of the notion of *supervision*. In the case of clustering, the data is partitioned into groups without any regard for the types of clusters we wish to find. In the case of classification, the *training* data are already partitioned into specific types of groups. Therefore, in addition to the $n \times d$ data matrix D , we have an $n \times 1$ array of labels denoted by $\bar{y}$. The i th entry in $\bar{y}$ corresponds to the i th row in the data matrix D , and the former is a categorical label defining a semantic name for the cluster (or *class*) to which the i th row of D belongs. In the case of the grocery example above, we might decide up front that we are interested in the classes $\mathcal{L} = \{\text{fruits, poultry, all else}\}$. Note that these classes might often be clustered in the data in terms of the similarity of the rows in the data matrix D , although this is not always necessary. For example, clusters that are clearly distinct might be located in a single class. Furthermore, it might be possible that other distinct clusters might exist that are corresponding to specific sub-categories within the *all else* label. This might be the case because the end-user (e.g., merchant) might not have any interest in identifying items in the *all else* category, whereas the other labels might help the merchant identify candidate customers for a promotion. Therefore, in the classification problem, the training data defines the clusters of *interest* with the use of examples. The actual segmentation of the rows is done on a separate $n_t \times d$ test data matrix D_t , in which the labels are not specified. Therefore, for each row of D_t , one needs to map it one of the labels from the set $\mathcal{L}$. This mapping is done with the use of a classification *model* that was constructed on the training data. The test data is *unseen* during the process of model construction, as the rows of D and D_t are not the same.

A common setting in classification is that the label set is *binary* and only contains two possible values. In such a case, it is common to use the label set $\mathcal{L}$ from $\{0, 1\}$ or from $\{-1, +1\}$. The goal is to *learn* the i th entry y_i in $\bar{y}$ as a function of the i th row $\bar{X}_i$ of D :

$$y_i \approx f(\bar{X}_i)$$

The function $f(\bar{X}_i)$ is often parameterized with a weight vector $\bar{W}$. Consider the following example of binary classification into the labels $\{-1, +1\}$:

$$y_i \approx f_{\bar{W}}(\bar{X}_i) = \text{sign}\{\bar{W} \cdot \bar{X}_i\}$$

Note that we have added a subscript to the function to indicate its parametrization. How does one compute $\bar{W}$? The key idea is to penalize any kind of mismatching between the *observed value* y_i and the *predicted value* $f(\bar{X}_i)$ with the use of carefully constructed *loss function*. Therefore, many machine learning models reduce to the following optimization problem:

$$\text{Minimize}_{\bar{W}} \sum_i \text{Mismatching between } y_i \text{ and } f_{\bar{W}}(\bar{X}_i)$$

Once the weight vector $\bar{W}$ has been computed by solving the optimization model, it is used to predict the value of the class variable y_i for instances in which the class variable is not known. Classification is also referred to as *supervised learning*, because it uses the training data to build a model that performs the classification of the test data. In a sense, the training data serves as the “teacher” providing supervision. The ability to use the knowledge in the training data in order to classify the examples in *unseen* test data is referred to as *generalization*. There is no utility in classifying the examples of the training data again, because their labels have already been observed.

Regression

The label in classification is also referred to as *dependent* variable, which is categorical in nature. In the regression modeling problem, the $n \times d$ training data matrix D is associated with an $n \times 1$ vector $\bar{y}$ of dependent variables, which are *numerical*. Therefore, the only difference from classification is that the array $\bar{y}$ contains numerical values (rather than categorical ones), and can therefore be treated as a vector. The dependent variable is also referred to as a *response variable*, *target variable*, or *regressand* in the case of regression. The independent variables are also referred to as *regressors*. Binary response variables are closely related to regression, and some models solve binary classification *directly* with the use of a regression model (by pretending that the binary labels are numerical). This is because binary values have the flexibility of being treated as either categorical or as numerical values. However, more than two classes like $\{\text{Red}, \text{Green}, \text{Blue}\}$ cannot be ordered, and are therefore different from regression.

The regression modeling problem is closely related to linear algebra, especially when a *linear optimization model* is used. In the linear optimization model, we use a d -dimensional column vector $\bar{W} = [w_1 \dots w_d]^T$ to represent the weights of the different dimensions. The i th entry y_i of $\bar{y}$ is obtained as the dot product of the i th row $\bar{X}_i$ of D and $\bar{W}$. In other words, the function $f(\cdot)$ to be learned by the optimization problem is as follows:

$$y_i = f(\bar{X}_i) = \bar{X}_i \bar{W}$$

One can also state this condition across all training instances using the full $n \times d$ data matrix D :

$$\bar{y} \approx D\bar{W} \tag{1.27}$$

Note that this is a matrix representation of n linear equations. In most cases, the value of n is much greater than d , and therefore, this is an *over-determined* system of linear equations. In over-determined cases, there is usually no solution for $\bar{W}$ that *exactly* satisfies this system. However, we can minimize the sum of squares of the errors to get as close to this goal as possible:

$$J = \frac{1}{2} \|D\bar{W} - \bar{y}\|^2 \tag{1.28}$$

On solving the aforementioned optimization problem, it will be shown in Chapter 4 that the solution $\bar{W}$ can be obtained as follows:

$$\bar{W} = (D^T D)^{-1} D^T \bar{y} \tag{1.29}$$

Then, for each row $\bar{Z}$ of the test data matrix D_t , the dot product of $\bar{W}^T$ and $\bar{Z}$ is the corresponding prediction of the real-valued dependent variable.

1.4.4 Outlier Detection

In the outlier detection problem, we have an $n \times d$ data matrix D , and we would like find rows of D that are very different from most of the other rows. This problem has a natural relationship of complementarity with the clustering problem, in which the aim is to find groups of similar rows. In other words, outliers are rows of D that do not naturally fit in with the other rows. Therefore, clustering methods are often used to find outliers. Matrix factorization methods are also used often for outlier detection. This book will introduce various outlier detection methods as applications of linear algebra and optimization.

1.5 Optimization for Machine Learning

Much of machine learning uses optimization in order to define *parameterized* models for learning problems. These models treat dependent variables as functions of independent variables, such as Equation 1.27. It is assumed that some examples are available containing observed values of both dependent and independent variables for training. These problems define *objective functions* or *loss functions*, which penalize differences between predicted and observed values of dependent variables (such as Equation 1.28). Therefore, the training phase of machine learning methods requires the use of optimization techniques.

In most cases, the optimization models are posed in minimization form. The most basic condition for optimality of the function $f(x_1, \dots, x_d)$ at $[x_1 \dots x_d]$ is that each *partial derivative* is 0:

$$\frac{\partial f(x_1, \dots, x_d)}{\partial x_r} = \lim_{\delta \rightarrow 0} \frac{f(x_1, \dots, x_r + \delta, \dots, x_d) - f(x_1, \dots, x_r, \dots, x_d)}{\delta} = 0, \quad \forall r$$

The basic idea is that the rate of change of the function in any direction is 0, or else one can move in a direction with negative rate of change to further improve the objective function. This condition is necessary, but not sufficient, for optimization. More details of relevant optimality conditions are provided in Chapter 4.

The d -dimensional vector of partial derivatives is referred to as the *gradient*:

$$\nabla f(x_1, \dots, x_d) = \left[\frac{\partial f(\cdot)}{\partial x_1} \dots \frac{\partial f(\cdot)}{\partial x_d} \right]^T$$

The gradient is denoted by the symbol ∇ , and putting it in front of a function refers to the vector of partial derivatives with respect to the argument.

1.5.1 The Taylor Expansion for Function Simplification

Many objective functions in machine learning are very complicated in comparison with the relatively simple structure of polynomial functions (which are much easier to optimize). Therefore, if one can approximate complex objective functions with simpler polynomials (even within restricted regions of the space), it can go a long way toward solving optimization problems in an iterative way.

The Taylor expansion expresses any smooth function as a polynomial (with an infinite number of terms). Furthermore, if we only want an approximation of the function in a small locality of the argument, a small number of polynomial terms (typically no more than 2 or 3) will often suffice. First, consider the univariate function $f(w)$. This function can be expanded about any point a in the domain of the function by using the following expansion:

$$f(w) = f(a) + (w - a)f'(a) + \frac{(w - a)^2 f''(a)}{2!} + \dots + \frac{(w - a)^r}{r!} \left[\frac{d^r f(w)}{d w^r} \right]_{w=a} + \dots$$

Here, $f'(a)$ is the first derivative of $f(w)$ at a , $f''(w)$ is the second derivative, and so on. Note that $f(w)$ could be an arbitrary function, such as $\sin(w)$ or $\exp(w)$, and the expansion expresses it as a polynomial with an infinite number of terms. The case of $\exp(w)$ is particularly simple, because the n th order derivative of $\exp(w)$ is itself. For example, $\exp(w)$ can be expanded about $w = 0$ as follows:

$$\exp(w) = \exp(0) + \exp(0)w + \exp(0)\frac{w^2}{2!} + \exp(0)\frac{w^3}{3!} + \dots + \exp(0)\frac{w^n}{n!} \dots \quad (1.30)$$

$$= 1 + w + \frac{w^2}{2!} + \frac{w^3}{3!} + \dots + \frac{w^n}{n!} \dots \quad (1.31)$$

In other words, the exponentiation function can be expressed as an infinite polynomial, in which the trailing terms rapidly shrink in size because $\lim_{n \rightarrow \infty} w^n/n! = 0$. For some functions like $\sin(w)$ and $\exp(w)$, the Taylor expansion converges to the true function by including an increasing number of terms (irrespective of the choice of w and a). For other functions like $1/w$ or $\log(w)$, a converging expansion exists in restricted ranges of w at any particular value of a . More importantly, the Taylor expansion almost always provides a very good approximation of any smooth function *near* $w = a$, and the approximation is exact at $w = a$. Furthermore, higher-order terms tend to vanish when $|w - a|$ is small, because $(w - a)^r/r!$ rapidly converges to 0 for increasing r . Therefore, one can often obtain good quadratic approximations of a function near $w = a$ by simply including the first three terms.

In practical settings like optimization, one is often looking to change the value w from the current point $w = a$ to a “nearby” point in order to improve the objective function value. In such cases, using only up to the quadratic term of the Taylor expansion about $w = a$ provides an excellent simplification in the neighborhood of $w = a$. In gradient-descent algorithms, one is often looking to move from the current point by a relatively small amount, and therefore lower-order Taylor approximations can be used to guide the steps in order to improve the polynomial approximation rather than the original function. It is often much easier to optimize polynomials than arbitrarily complex functions.

One can also generalize the Taylor expansion to multivariable functions $F(\bar{w})$ with d -dimensional arguments of the form $\bar{w} = [w_1 \dots w_d]^T$. The Taylor expansion of the function $F(\bar{w})$ about $\bar{w} = \bar{a} = [a_1 \dots a_d]^T$ can be written as follows:

$$\begin{aligned} F(\bar{w}) = F(\bar{a}) &+ \sum_{i=1}^d (w_i - a_i) \left[\frac{\partial F(\bar{w})}{\partial w_i} \right]_{\bar{w}=\bar{a}} + \sum_{i=1}^d \sum_{j=1}^d \frac{(w_i - a_i)(w_j - a_j)}{2!} \left[\frac{\partial^2 F(\bar{w})}{\partial w_i \partial w_j} \right]_{\bar{w}=\bar{a}} + \\ &+ \sum_{i=1}^d \sum_{j=1}^d \sum_{k=1}^d \frac{(w_i - a_i)(w_j - a_j)(w_k - a_k)}{3!} \left[\frac{\partial^3 F(\bar{w})}{\partial w_i \partial w_j \partial w_k} \right]_{\bar{w}=\bar{a}} + \dots \end{aligned}$$

In the multivariable case, we have $O(d^2)$ second-order interaction terms, $O(d^3)$ third-order interaction terms, and so on. One can see that the number of terms becomes unwieldy very quickly. Luckily, we rarely need to go beyond second-order approximations in practice. Furthermore, the above expression can be rewritten using the gradients and matrices compactly. For example, the second-order approximation can be written in vector form as follows:

$$F(\bar{w}) \approx F(\bar{a}) + [\bar{w} - \bar{a}]^T \nabla F(\bar{w}) + [\bar{w} - \bar{a}]^T H(\bar{a}) [\bar{w} - \bar{a}]$$

Here, $\nabla F(\bar{w})$ is the gradient, and $H(\bar{a}) = [h_{ij}]$ is the $d \times d$ matrix of all second-order derivatives of the following form:

$$h_{ij} = \left[\frac{\partial^2 F(\bar{w})}{\partial w_i \partial w_j} \right]_{\bar{w}=\bar{a}}$$

A third-order expansion would require the use of a *tensor*, which is a generalization of the notion of a matrix. The first- and second-order expansions will be used frequently in this book for developing various types of optimization algorithms, such as the Newton method.

Problem 1.5.1 (Euler Identity) *The Taylor series is valid for complex functions as well. Use the Taylor series to show the Euler identity $e^{i\theta} = \cos(\theta) + i \sin(\theta)$.*

1.5.2 Example of Optimization in Machine Learning

An example of a parameterized model discussed in an earlier section is that of linear regression, in which we want to determine a d -dimensional vector $\bar{W} = [w_1 \dots w_d]^T$ so that we can predict the n -dimensional dependent variable vector $\bar{y}$ as a function $\bar{y} = D\bar{W}$ of the $n \times d$ matrix D of the observed values. In order to minimize the difference between predicted and observed values, the following objective function is minimized:

$$J = \frac{1}{2} \|D\bar{W} - \bar{y}\|^2 \quad (1.32)$$

Here, D is an $n \times d$ data matrix, whereas $\bar{y}$ is an n -dimensional column vector of dependent variables. Therefore, this is a simple optimization problem in d parameters. Finding the optimal solution requires techniques from differential calculus. The simplest approach is to set the partial derivative with respect to each parameter w_i to 0, which provides a necessary (but not sufficient) condition for optimality:

$$\frac{\partial J}{\partial w_i} = 0, \quad \forall i \in \{1 \dots d\} \quad (1.33)$$

The partial derivatives can be shown to be the following (cf. Section 4.7 of Chapter 4):

$$\left[\frac{\partial J}{\partial w_1} \dots \frac{\partial J}{\partial w_d} \right]^T = D^T D\bar{W} - D^T \bar{y} \quad (1.34)$$

For certain types of *convex objective functions* like linear regression, setting the vector of partial derivatives to the zero vector is both necessary and sufficient for minimization (cf. Chapters 3 and 4). Therefore, we have $D^T D\bar{W} = D^T \bar{y}$, which yields the following:

$$\bar{W} = (D^T D)^{-1} D^T \bar{y} \quad (1.35)$$

Linear regression is a particularly simple problem because the optimal solution exists in *closed form*. However, in most cases, one cannot solve the resulting optimality conditions in such a form. Rather, the approach of *gradient-descent* is used. In gradient descent, we use a computational algorithm of initializing the parameter set $\bar{W}$ randomly (or a heuristically chosen point), and then change the parameter set in the direction of the negative derivative of the objective function. In other words, we use the following updates repeatedly with step-size α , which is also referred to as the *learning rate*:

$$[w_1 \dots w_d]^T \Leftarrow [w_1 \dots w_d]^T - \alpha \left[\frac{\partial J}{\partial w_1} \dots \frac{\partial J}{\partial w_d} \right]^T = \bar{W} - \alpha [D^T D\bar{W} - D^T \bar{y}] \quad (1.36)$$

The d -dimensional vector of partial derivatives is referred to as the *gradient vector*, and it defines an instantaneous direction of best rate of improvement of the objective function at the current value of the parameter vector $\bar{W}$. The gradient vector is denoted by $\nabla J(\bar{W})$:

$$\nabla J(\bar{W}) = \left[\frac{\partial J}{\partial w_1} \dots \frac{\partial J}{\partial w_d} \right]^T$$

Therefore, one can succinctly write gradient descent in the following form:

$$\overline{W} \leftarrow \overline{W} - \alpha \nabla J(\overline{W})$$

The size of the step is defined by the learning rate α . Note that the best rate of improvement is only over a step of infinitesimal size, and does not hold true for larger steps of finite size. Since the gradients change on making a step, one must be careful not to make steps that are too large or else the effects might be unpredictable. These updates are repeatedly executed to *convergence*, when further improvements become too small to be useful. Such a situation will occur when the gradient vector contains near-zero entries. Therefore, this computational approach will also (eventually) reach a solution approximately satisfying the optimality conditions of Equation 1.33. As we will show in Chapter 4, the gradient descent method (and many other optimization algorithms) can be explained with the use of the Taylor expansion.

Using gradient descent for optimization is a tricky exercise, because one does not always converge to an optimal solution for a variety of reasons. For example, even the wrong step-size, α , might result in unexpected numerical overflows. In other cases, one might terminate at suboptimal solutions, when the objective function contains multiple minima relative to specific local regions. Therefore, there is a significant body of work on designing optimization algorithms (cf. Chapters 4, 5, and 6).

1.5.3 Optimization in Computational Graphs

Many machine learning problems can be represented as the process of learning a function of the inputs that matches the observed variables in the data. For example, the least-squares optimization problem can be represented as the following sequence of operations:

Input (d variables) $\Rightarrow$ Dot product with parameter vector $\overline{W} \Rightarrow$ Prediction $\Rightarrow$ Squared loss

A graphical representation of these types of operations on the inputs is presented in Figure 1.5(a). This model has d input nodes containing the features $x_1 \dots x_d$ of the data, and a single (computational) output node creating the dot product $\sum_{i=1}^d w_i x_i$. The weights $[w_1 \dots w_d]$ are associated with the edges. Therefore, each node computes a function of its inputs, and the edges are associated with the parameters to be learned. By choosing a more complex topology of the computational graph with more nodes, one can create more powerful models, which often do not have direct analogs in traditional machine learning

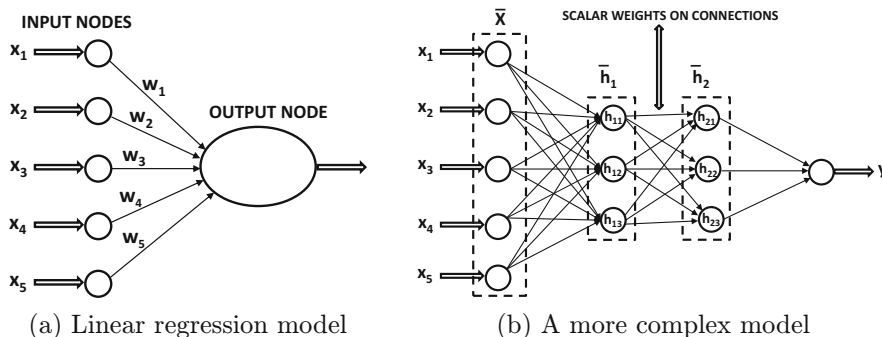


Figure 1.5: The computational graph view of machine learning

(cf. Figure 1.5(b)). Each node of this graph can compute a function of its incoming nodes and the edge parameters. The overall function is potentially extremely complex, and often cannot be expressed compactly in closed form (like the simple relationship $y = \sum_{i=1}^d w_i x_i$ in a linear regression model). A model with many layers of nodes is referred to as a *deep learning model*. Such models can learn complex, nonlinear relationships in the data.

How does one compute gradients with respect to edge parameters in computational graphs? This is achieved with the use of a technique referred to as *backpropagation*, which will be introduced in Chapter 11. The backpropagation algorithm yields exactly the same gradient as is computed in traditional machine learning. For example, since Figure 1.5(a) models linear regression, the backpropagation algorithm will yield exactly the same gradient as computed in the previous section. The main difference is that the backpropagation algorithm will also be able to compute gradients in more complex cases like Figure 1.5(b). Almost all the well-known machine learning models (based on gradient descent) can be represented as relatively simple computational graphs. Therefore, computational graphs are extremely powerful abstractions, as they include traditional machine learning as special cases. We will discuss the power of such models and the associated algorithms in Chapter 11.

1.6 Summary

Linear algebra and optimization are intimately related because many of the basic problems in linear algebra, such as finding the “best” solution to an over-determined system of linear equations, are solved using optimization techniques. Many optimization models in machine learning can also be expressed as objective functions and constraints using matrices/vectors. A useful technique that is used in many of these optimization problems is to decompose these matrices into simpler matrices with specific algebraic/geometric properties. In particular, the following two types of decomposition are commonly used in machine learning:

- Any square and invertible matrix A can be decomposed into a product of elementary matrix operators. If the matrix A is not invertible, it can still be decomposed with a *relaxed* definition of matrix operators, which are allowed to be non-invertible.
- Any square matrix A can be decomposed into a product of two rotation matrices and one scaling (diagonal) matrix in the particular order of rotation, scaling, and rotation. This idea is referred to as *singular value decomposition* (cf. Chapter 7).

An alternative view of machine learning expresses predictions as *computational graphs*; this idea also forms the basis for the field of *deep learning*.

1.7 Further Reading

Several basic books on linear algebra are available, such as those by Strang [122, 123], David Lay [77], and Hoffman-Kunze [62]. These books are, however, generic books on linear algebra, and the focus is not specifically on machine learning topics. Some recent books have focused on a machine learning perspective [23, 119, 122, 125]. The classic matrix computation book by Golub and Van Loan [52] provides an overview of fundamental numerical algorithms. A closely related field to linear algebra is that of optimization. Several books are available on optimization from a generic perspective [10, 15, 16, 22, 99], whereas others focus on machine learning [1–4, 18, 19, 39, 46, 53, 56, 85, 94, 95].

1.8 Exercises

- For any two vectors $\bar{x}$ and $\bar{y}$, which are each of length a , show that (i) $\bar{x} - \bar{y}$ is orthogonal to $\bar{x} + \bar{y}$, and (ii) the dot product of $\bar{x} - 3\bar{y}$ and $\bar{x} + 3\bar{y}$ is negative.
- Consider a situation in which you have three matrices A , B , and C , of sizes 10×2 , 2×10 , and 10×10 , respectively.
 - Suppose you had to compute the matrix product ABC . From an efficiency perspective, would it computationally make more sense to compute $(AB)C$ or would it make more sense to compute $A(BC)$?
 - If you had to compute the matrix product CAB , would it make more sense to compute $(CA)B$ or $C(AB)$?
- Show that if a matrix A satisfies $A = -A^T$, then all the diagonal elements of the matrix are 0.
- Show that if we have a matrix satisfying $A = -A^T$, then for any column vector $\bar{x}$, we have $\bar{x}^T A \bar{x} = 0$.
- Suppose we have an $n \times n$ matrix A that can be written as $A = D^T$ for some $n \times d$ matrix D . Show that $\bar{x}^T A \bar{x} \geq 0$ for any n -dimensional column vector $\bar{x}$.
- Show that the matrix product AB remains unchanged if we scale the i th column of A and the i th row of B by respective factors that are inverses of each other.
- Show that any matrix product AB can be expressed in the form $A'\Delta B'$, where A' is a matrix in which the sum of the squares of the entries in each column is 1, B' is a matrix in which the sum of the squares of the entries in each row is 1, and Δ is an appropriately chosen diagonal matrix with nonnegative entries on the diagonal.
- Discuss how a permutation matrix can be converted to the identity matrix using at most d elementary row operations of a single type. Use this fact to express A as the product of at most d elementary matrix operators.
- Suppose that you reorder all the columns of an invertible matrix A using some random permutation, and you know A^{-1} for the original matrix. Show how you can (simply) compute the inverse of the reordered matrix from A^{-1} without having to invert the new matrix from scratch. Provide an argument in terms of elementary matrices.
- Suppose that you have approximately factorized an $n \times d$ matrix D as $D \approx UV^T$, where U is an $n \times k$ matrix and V is a $d \times k$ matrix. Show how you can derive an infinite number of alternative factorizations $U'V'^T$ of D , which satisfy $UV^T = U'V'^T$.
- Either prove each of the following statements or provide a counterexample:
 - The order in which you apply two elementary row operations to a matrix does not affect the final result.
 - The order in which you apply an elementary row operation and an elementary column operation does not affect the final result.

It is best to think of these problems in terms of elementary matrix operations.

12. Discuss why some power of a permutation matrix is always the identity matrix. [Hint: Think in terms of the finiteness of the number of permutations.]
13. Consider the matrix polynomial $\sum_{i=0}^t a_i A^i$. A straightforward evaluation of this polynomial will require $O(t^2)$ matrix multiplications. Discuss how you can reduce the number of multiplications to $O(t)$ by rearranging the polynomial.
14. Let $A = [a_{ij}]$ be a 2×2 matrix with $a_{12} = 1$, and 0s in all other entries. Show that $A^{1/2}$ does not exist even after allowing complex-valued entries.
15. **Parallelogram law:** The parallelogram law states that the sum of the squares of the sides of a parallelogram is equal to the sum of the squares of its diagonals. Write this law as a vector identity in terms of vectors $\overline{A}$ and $\overline{B}$ of Figure 1.1. Now use vector algebra to show why this vector identity must hold.
16. Write the first four terms of the Taylor expansion of the following univariate functions about $x = a$: (i) $\log_e(x)$; (ii) $\sin(x)$; (iii) $1/x$; (iv) $\exp(x)$.
17. Use the multivariate Taylor expansion to provide a quadratic approximation of $\sin(x+y)$ in the vicinity of $[x, y] = [0, 0]$. Confirm that this approximation loses its accuracy with increasing distance from the origin.
18. Consider a case where a $d \times k$ matrix P is initialized by setting all values randomly to either -1 or $+1$ with equal probability, and then dividing all entries by $\sqrt{d}$. Discuss why the columns of P will be (roughly) mutually orthogonal for large values of d of the order of 10^6 . This trick is used frequently in machine learning for rapidly generating the random projection of an $n \times d$ data matrix D as $D' = DP$.
19. Consider the perturbed $d \times d$ matrix $A_\epsilon = A + \epsilon B$, where the value of ϵ is small. Show the following useful approximation for approximating A_ϵ^{-1} from A^{-1} :

$$A_\epsilon^{-1} \approx A^{-1} - \epsilon A^{-1} B A^{-1}$$

20. Suppose that you have a 5×5 matrix A , in which the rows/columns correspond to people in a social network in the order John, Mary, Jack, Tim, and Robin. The entry (i, j) corresponds to the number of times person i sent a message to person j . Define a matrix P , so that PAP^T contains the same information, but with the rows/columns in the order Mary, Tim, John, Robin, and Jack.
21. Suppose that the vectors $\overline{x}$, $\overline{y}$, and $\overline{x} - \overline{y}$ have lengths 2, 3, and 4, respectively. Find the length of $\overline{x} + \overline{y}$ using only vector algebra (and no Euclidean geometry).
22. Show that the inverse of a symmetric matrix is symmetric.
23. Let $A_1, A_2, \dots, A_d$ be $d \times d$ matrices that are strictly upper triangular. Then, the product of $A_1, A_2, \dots, A_d$ is the zero matrix.
24. **Apollonius's identity:** Let ABC be a triangle, and AD be the median from A to BC. Show the following using only vector algebra and no Euclidean geometry:

$$AB^2 + AC^2 = 2(AD^2 + BD^2)$$

[Hint: Orient your triangle properly with respect to the origin.]

- 25. Sine law:** Express the sine of the *interior* angle between $\bar{a}$ and $\bar{b}$ (i.e., the angle not greater than 180 degrees) purely in terms of $\bar{a} \cdot \bar{a}$, $\bar{b} \cdot \bar{b}$, and $\bar{a} \cdot \bar{b}$. You are allowed to use $\sin^2(x) + \cos^2(x) = 1$. Consider a triangle, two sides of which are the vectors $\bar{a}$ and $\bar{b}$. The *opposite* angles to these vectors are A and B , respectively. Show the following using only vector algebra and no Euclidean geometry:

$$\frac{\|\bar{a}\|}{\sin(A)} = \frac{\|\bar{b}\|}{\sin(B)}$$

- 26. Trigonometry with vector algebra:** Consider a unit vector $\bar{x} = [1, 0]^T$. The vector $\bar{v}_1$ is obtained by rotating $\bar{x}$ counter-clockwise by angle θ_1 , and $\bar{v}_2$ is obtained by rotating $\bar{x}$ clockwise by θ_2 . Use the rotation matrix to obtain the coordinates of unit vectors $\bar{v}_1$ and $\bar{v}_2$, and then show the following well-known trigonometric identity:

$$\cos(\theta_1 + \theta_2) = \cos(\theta_1)\cos(\theta_2) - \sin(\theta_1)\sin(\theta_2)$$

- 27. Coordinate geometry with matrix algebra:** Consider the two lines $y = 3x + 4$ and $y = 5x + 2$ in the 2-dimensional plane. Write the equations in matrix form for appropriately chosen A and $\bar{b}$:

$$A \begin{bmatrix} x \\ y \end{bmatrix} = \bar{b}$$

Find the intersection coordinates (x, y) of the two lines by inverting matrix A .

- 28.** Use the matrix inversion lemma to invert a 10×10 matrix with 1s in each entry other than the diagonal entries, which contain the value 2.
- 29. Solid geometry with vector algebra:** Consider the origin-centered hyperplane in 3-dimensional space that is defined by the equation $z = 2x + 3y$. This equation has infinitely many solutions, all of which lie on the plane. Find two solutions that are not multiples of one another and denote them by the 3-dimensional column vectors $\bar{v}_1$ and $\bar{v}_2$, respectively. Let $V = [\bar{v}_1, \bar{v}_2]$ be a 3×2 matrix with columns $\bar{v}_1$ and $\bar{v}_2$. Geometrically describe the set of all vectors that are linear combinations of $\bar{v}_1$ and $\bar{v}_2$ with real coefficients c_1 and c_2 :

$$\mathcal{V} = \left\{ V \begin{bmatrix} c_1 \\ c_2 \end{bmatrix} : c_1, c_2 \in \mathcal{R} \right\}$$

Now consider the point $[x, y, z]^T = [2, 3, 1]^T$, which does not lie on the above hyperplane. We want to find a point $\bar{b}$ on the hyperplane for which $\bar{b}$ is as close to $[2, 3, 1]^T$ as possible. How is the vector $\bar{b} - [2, 3, 1]^T$ geometrically related to the hyperplane? Use this fact to show the following condition on $\bar{b}$:

$$V^T \left(\bar{b} - \begin{bmatrix} 2 \\ 3 \\ 1 \end{bmatrix} \right) = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Find a way to eliminate the 3-variable vector $\bar{b}$ from the above equation and replace with the 2-variable vector $\bar{c} = [c_1, c_2]^T$ instead. Substitute numerical values for entries in V and find $\bar{c}$ and $\bar{b}$ with a 2×2 matrix inversion.

- 30.** Let A and B be two $n \times d$ matrices. One can partition them columnwise as $A = [A_1, A_2]$ and $B = [B_1, B_2]$, where A_1 and B_1 are $n \times k$ matrices containing the first k columns of A and B , respectively, in the same order. Let A_2 and B_2 contain the remaining columns. Show that the matrix product AB^T can be expressed as follows:

$$AB^T = A_1B_1^T + A_2B_2^T$$

- 31. Matrix centering:** In machine learning, a common centering operation of an $n \times n$ similarity matrix S is the update $S \leftarrow (I - U/n)S(I - U/n)$, where U is an $n \times n$ matrix of 1s. Use the associative property of matrix multiplication to implement this update efficiently. [Hint: Express U as a product of smaller matrices.]
- 32. Energy preservation in orthogonal transformations:** Show that if A is an $n \times d$ matrix and P is a $d \times d$ orthogonal matrix, then we have $\|AP\|_F = \|A\|_F$.
- 33. Tight sub-multiplicative case:** Suppose that $\bar{u}$ and $\bar{v}$ are column vectors (of not necessarily the same dimensionality). Show that the matrix $\bar{u}\bar{v}^T$ created from the outer product of $\bar{u}$ and $\bar{v}$ has Frobenius norm of $\|\bar{u}\| \|\bar{v}\|$.
- 34. Frobenius orthogonality and Pythagorean theorem:** Two $n \times d$ matrices A and B are said to be Frobenius orthogonal if the sum of entry-wise products of their corresponding elements is zero [i.e., $\text{tr}(AB^T) = 0$]. Show the following:

$$\|A + B\|_F^2 = \|A\|_F^2 + \|B\|_F^2$$

- 35.** Let $\bar{x}$ and $\bar{y}$ be two orthogonal column vectors of dimensionality n . Let $\bar{a}$ and $\bar{b}$ be two arbitrary d -dimensional column vectors. Show that the outer products $\bar{x}\bar{a}^T$ and $\bar{y}\bar{b}^T$ are Frobenius orthogonal (see Exercise 34 for definition of Frobenius orthogonality).
- 36.** Suppose that a sequence of row and column operations is performed on a matrix. Show that as long as the ordering among row operations and the ordering among column operations is maintained, the way in which the row sequence and column sequence are merged does not change the final result matrix. [Hint: Use operator matrices.]
- 37.** Show that any orthogonal upper-triangular matrix is a diagonal matrix.
- 38.** Consider a set of vectors $\bar{x}_1 \dots \bar{x}_n$, which are known to be *unit normalized*. You do not have access to the vectors, but you are given all pairwise squared Euclidean distances in the $n \times n$ matrix Δ . Discuss why you can derive the $n \times n$ pairwise dot product matrix by adding 1 to each entry of the matrix $-\frac{1}{2}\Delta$.
- 39.** We know that every matrix commutes with its inverse. We want to show a generalization of this result. Consider the polynomial functions $f(A)$ and $g(A)$ of the square matrix A , so that $f(A)$ is invertible. Show the following commutative property:

$$[f(A)]^{-1}g(A) = g(A)[f(A)]^{-1}$$

- 40.** Give an example of a 2×2 matrix A and a polynomial function $f(\cdot)$, so that A is invertible, but $f(A)$ is not invertible. Give an example of a matrix A , so that A is not invertible, but $f(A)$ is invertible. Note that the constant term in the polynomial corresponds to a multiple of the identity matrix.

41. Let A be a rectangular matrix and $f(\cdot)$ be a polynomial function. Show that $A^T f(AA^T) = f(A^T A)A^T$. Assuming invertibility of $f(AA^T)$ and $f(A^T A)$, show:

$$[f(A^T A)]^{-1} A^T = A^T [f(AA^T)]^{-1}$$

Interpret the push-through identity as a special case of this result.

42. Discuss why one cannot generalize the formula for the scalar binomial expansion $(a+b)^n$ to the matrix expansion $(A+B)^n$. Also discuss why generalization is possible in cases where $B = f(A)$ for some polynomial function $f(\cdot)$.
43. Suppose that A is a $d \times d$ matrix satisfying $A^4 = 0$. Derive an algebraic expression for $(I + A)^{-1}$ as a matrix polynomial in A .
44. Compute the inverse of the following triangular matrix by expressing it as the sum of two carefully chosen matrices (cf. Section 1.2.5):

$$A = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 1 & 3 & 1 \end{bmatrix}$$

45. Express a $d \times d$ matrix M of 1s as the outer product of two d -dimensional vectors. Use the matrix inversion lemma to compute an algebraic expression for $(I + M)^{-1}$.
46. Show that if A and B commute, the matrix polynomials $f(A)$ and $g(B)$ commute.
47. Show that if invertible matrices A and B commute, A^k and B^s commute for all integers $k, s \in [-\infty, \infty]$. Show the result of Exercise 46 for an extended definition of “polynomials” with both positive and negative integer exponents included.
48. Let $U = [u_{ij}]$ be an upper-triangular $d \times d$ matrix. What are the diagonal entries of the matrix polynomial $f(U)$ as scalar functions of the matrix entries u_{ij} ?
49. **Inverses behave like matrix polynomials:** The *Cayley-Hamilton theorem* states that a finite-degree polynomial $f(\cdot)$ always exists for any matrix A satisfying $f(A) = 0$. Use this fact to prove that the inverse of A is also a finite-degree polynomial.
50. Derive the inverse of a 3×3 row addition operator by inverting the sum of matrices.
51. For any non-invertible matrix A , show that the infinite summation $\sum_{k=0}^{\infty} (I - A)^k$ cannot possibly converge to a finite matrix. Give two examples to show that if A is invertible, the summation might or might not converge.
52. The chapter shows that the product, $A_1 A_2 \dots A_k$, of invertible matrices is invertible. Show the converse that if the product $A_1 A_2 \dots A_k$ of square matrices is invertible, each matrix A_i is invertible. [Hint: You need only the most basic results discussed in this chapter for the proof.]
53. Show that if a $d \times d$ diagonal matrix Δ with distinct diagonal entries $\lambda_1 \dots \lambda_d$ commutes with A , then A is diagonal.
54. What fraction of 2×2 binary matrices with 0-1 entries are invertible?